

On Electronic Health Record Synthetic Data Generation with an Interactive Clinical Analysis Interface

Jaagat Prashar

CS 195

1 Abstract

In this report, I provide a comprehensive overview of the research progress that I have conducted thus far under the supervision of _____ and _____ as part of CS 195. Concisely, the goals of such research lie in generating synthetic patient electronic health records (EHR) that emphasize patient trajectories over time, ensuring that the synthetic data generated is of high fidelity and quality, and developing a seamless user interface that enables the visualization of relevant clinical trends and information pertinent to a given patient. The goal of producing synthetic data is driven by the feasibility for synthetic data to be leveraged for downstream ML tasks and analyses. A particular application lies in the recent advancements of large-language models (LLMs), which have revealed impressive capabilities in domains such as reasoning and contextual awareness (RAG, chain-of-thought, etc.), thereby making them suitable to apply on medical data. Through this research project, we aim to understand how we can 1) rigorously process, clean, and prepare real medical training data for synthetic data generation, 2) leverage deep learning techniques such as GANs and diffusion models to generate synthetic EHR data to protect the privacy of patients, 3) evaluate the quality of synthetic data by comparing it to real data through statistical/distribution comparisons, correlation matrices, PCA, UMAP, ML prediction tasks, etc., 4) utilize techniques such as RAG and prompt engineering to elicit context-dependent medical information from medical data, and 5) develop a seamless user interface to promote the ability to understand and reason through EHR timelines and trends. This report summarizes my work in conducting literature reviews on papers relevant to LLMs for medical data analysis and synthetic EHR data, generating synthetic EHR data, evaluating synthetic EHR data in reference to real data, learning about ML techniques such as RAG and prompt engineering, and developing a user interface to visualize patient trends over time. I explored SOTA frameworks such as CTGAN, SynLongEHR, GAN Tutorials, medGAN, and ScoEHR with datasets such as Medical Information Mart for Intensive Care (MIMIC-IV) and EHRSHOT. Since this was my first quarter of research, a large emphasis on experimentation, exploration, and learning, which I outline in my work. All synthetically generated data can be found [here](#).

2 Introduction

Artificial intelligence (AI) has significantly transformed the way that humanity interacts with data on a day-to-day basis. Whether it's powerful large language models (LLMs) that support billions of queries on ChatGPT or vision language action models that enable self-driving vehicles to directly map sensory input to logical actions, AI leverages vast amounts of diverse data to enable more informed decisions. In fields such as robotics and healthcare, we are now able to leverage data at a much larger scale than ever before and thus answer pressing scientific questions. For e.g., an important question that has revealed the capabilities of deep learning in biology is "Given protein amino acid sequences, how can we predict the structure of a protein?" AlphaFold[1], an AI program developed by Google DeepMind, accomplishes exactly that. Through rigorous training on publicly available protein datasets such as the Protein Data Bank (PDB) and UniProt, AlphaFold has been able to predict "over 200 million protein structures – nearly all catalogues of proteins known to science." [2] On the other hand, in the domain of robotics, companies such as 1X and Physical Intelligence[3] are working on developing general-purpose robots that leverage pre-trained VLMs to interpret and reason through sensory input in the real world. To this end, AI has also unlocked exciting fields such as spatial intelligence. For example, World Labs is a spatial intelligence company leveraging large world models (LWMs) to reason through the 3D world, expanding the capabilities of humanity to create and engineer.

Ultimately, these breakthroughs, whether in healthcare or robotics, underscore an important trend: the ability of modern AI systems to interpret vast and diverse amounts of data to support decision-making and prediction abilities. In this paper, I will be focusing on the various applications of AI to the healthcare field. Namely, how we can use deep learning to generate synthetic electronic health records (EHRs). Today, we can

leverage the powerful capabilities of AI architectures (e.g., diffusion models, variational autoencoders, generative adversarial networks, to name a few) to support pressing problems such as synthetic data generation.

We largely decided to focus on EHRs since a key limitation lies in modern clinical settings, where EHRs are more often than not unstructured and overwhelming to comprehend in volume and diversity [4]. Thus, doctors are presented with the challenge of interpreting vast amounts of diverse patient data on a daily basis. Such data ranges from clinical notes to vital signs to a patient’s medication history. With high dimensional datasets, we can leverage AI to infer trends that may not be easily available to the human brain. A major bottleneck in supporting AI experiments with EHR data is the need for synthetic data. Given the privacy restrictions (required training, legal paperwork, delays, etc.) that are attached to real healthcare data, it has become increasingly important to generate high fidelity synthetic patient data to permit downstream AI research (prediction tasks, for example).

Another fundamental gap to bridge is the availability of vast amounts of data, where AI can support humans in making rich inferences and analyses [5]. For instance, with 50 features/characteristics for patients, training a model to understand relationships can unlock new insights that may not be clear at first sight to a human. Thus, in this project, we aim to address this gap and more importantly, the need to use synthetic patient data for downstream ML tasks (since we can’t conduct ML tasks on sensitive personal data) by conducting synthetic data generation. To this end, we also work on developing an EHR dashboard user (discussed later) interface to support the way doctors can interact with such large patient datasets.

In developing synthetic EHR data, deep learning techniques such as generative adversarial networks (GANs), variational autoencoders/autoencoders, and diffusion models have emerged as promising approaches. In the paper, “Synthetic data generation methods in healthcare: A review on open-source tools and methods,” [6] the authors highlight the four stages of data generation: 1. Data acquisition, 2. Data preparation, 3. Data modeling, and 4. Data quality evaluation, which I used to guide my research. More importantly to the topic of leveraging deep learning architectures, the paper reveals various workflows and summarizes how GANs and VAEs are reputable and commonly used, with diffusion models also showing promising results, but not as frequently used as GANs and VAEs. In this paper, I implement the following frameworks: CTGAN, SynLongEHR, a classical GAN [7], medGAN, and ScoEHR. CTGAN [8] is a conditional GAN for generating synthetic tabular data. SynLongEHR [9] leverages two models in their workflow: a GAN, specifically DoppelGANger [10], and a conditional probabilistic auto-regressive network (CPAR) [11]. medGAN[12] leverages an autoencoder and GAN to produce multi-label discrete patient data. Finally, ScoEHR [13], although it does not explicitly use a variational autoencoder, leverages an autoencoder and a continuous time-diffusion model.

For the user interface, the ultimate end goal lies in enabling physicians to interact with patient data much more confidently and effectively than ever before (with AI-insights offering fresh perspectives that a physician may miss simply due to vast amounts of data). Specifically, we envision an AI system that is capable of the following: 1) interpreting a patient’s health trajectory over time with respect to clinical notes, vital signs, etc. 2) identifying causal trends/relationships in understanding a patient’s timeline (i.e., supporting a diagnosis through a temporal analysis of symptoms, treatments, and outcomes), and 3) minimizing delayed or potentially inaccurate medical decisions by mitigating interpretations with confounding variables and thus enabling more accurate recommendations. In short, we view this project as a checkpoint for contributing to the scientific research community in the spaces of synthetic data generation and EHR-interface development, with the goal of enabling more accurate clinical decisions.

3 Literature Review and Self-Learning

Throughout my research experience, I made an active effort to consistently explore and read papers relevant to state-of-the-art (SOTA) frameworks for interpreting and generating synthetic healthcare data, as well as understand how LLMs/agents can interact with healthcare data. In this section, I describe 9 relevant papers I had come across in my initial weeks in detail. Note that in future sections of my work, I inevitably discuss further papers/implementations. Thus, I avoid redundancy and do not mention them here. To allow for a more neatly formatted report, I have placed the content for this section at the end of this paper (please see Section 9).

4 Methodology - Implementation and Results

In understanding the question of how we can generate synthetic EHRs, it is evident that we first need access to a high-quality and realistic dataset with diverse patient data. Why is this important? The answer intuitively lies in the fact that we would want to preserve the distributions and observations that are present in a realistic healthcare dataset. Moreover, if we were to perform downstream tasks with such synthetically generated healthcare/EHR data (for instance, an ML prediction task for mortality or x disease), such a prediction must

be founded on reliable, realistic healthcare trends. The ground truth for reliable trends is present in real, deidentified healthcare datasets.

Throughout the project, I had placed a large emphasis on leveraging the Medical Information Mart for Intensive Care (MIMIC)-IV dataset. MIMIC, an EHR dataset made available by the research at Beth Israel Deaconess Medical Center (BIDMC) and MIT, is a “contemporary EHR record dataset” that consists of rich medical information collected between 2008 and 2019 [14].

The most recent version of the MIMIC dataset is indeed the 4th version (MIMIC-IV v3.1), which was the most widely adopted version for the analyses/results present in this section. The MIMIC-IV dataset is hosted on PhysioNet and its structure is fundamentally divided into two folders: hosp and icu. The hosp folder consists of “546,028 unique hospitalizations” (admission IDs in the dataset) for a total of “223,452 unique individuals” (subject IDs in the dataset). The measurements in this folder are measurements that were gathered during the duration of a patient’s hospital visit (e.g., laboratory measurements, medication prescriptions, billing information, etc.). On the other hand, the icu folder consists of “94,458 ICU stays” for “65,366” unique patients [15]. Since this dataset inherently consists of strict supervision for a patient’s medical progress in the intensive care unit, the data in this icu folder is often measured at closer time intervals (e.g, hours to days), providing rich medical information on each patient’s progress. According to the MIMIC-III dataset, the preceding version, the median length of an ICU stay was 2.1 days [16]. Due to the convenience at which the MIMIC-IV dataset can be accessed (i.e., quick online training) and the richness of medical data provided in both the hosp and icu modules, this dataset was widely implemented for several data processing and ML workflows. Note that in reproducing results for various papers, I had defaulted to leveraging older versions for compatibility purposes and at times, had also adapted the code to fit the MIMIC-IV dataset instead. Furthermore, in comparison to other datasets such as HiRID (ICU specific) [17] and eICU-CRD (ICU specific) [18], the MIMIC dataset stands out as a reliable choice due to its generality across multiple domains (as opposed to a limited focus) [14].

In researching the various deep learning techniques that researchers have used for synthetic data generation, I came across the following paper surveying SOTA approaches for synthetic data generation for a variety of data formats (tabular data, time-series data, image data, etc.) and the necessary technical prerequisites: “Synthetic data generation methods in healthcare: A review on open-source tools and methods” [6].

After reading this paper, self-studying deep learning architectures such as GANs and VAEs, I had begun to explore several papers on synthetic data generation through such architectures, 5 of which I have produced the results for below (using MIMIC and EHRSHOT): CTGAN, SynLongEHR, a GAN tutorial (as described in the initial report), MedGAN, and ScoEHR.

Below, I outline a concise understanding of my learnings of GANs and VAEs, as well as their relevance to synthetic data generation. For learning about these two architectures, I primarily relied on YouTube tutorials (this video on GANs and this video on VAEs offered a gentle introduction) and the papers on workflows I was implementing themselves. Note that while all of these implementations don’t directly apply VAEs, medGAN and ScoEHR apply an autoencoder. Nonetheless, I found my learnings about VAEs to be relevant for understanding these implementations and the problem of synthetic data generation.

GANs: A simple analogy discussed in the video that I enjoyed when learning about GANs was the presence of a counterfeiter attempting to produce fake money insofar that it could fool the police to think that the fake money is “real money.” In this analogy, the generator is the counterfeiter and the discriminator is the police. These two subjects are adversaries, for the generator becomes better at making fake data, whereas the discriminator becomes better at spotting fake data (when the police is confused and can’t tell the difference, the generator is creating valuable, realistic data that mimics the real data well). Fundamentally, GANs have a generator and discriminator. GANs are built on the minimax objective.

$$\max_D V(D) = \mathbb{E}_{\mathbf{x} \sim p_{data}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_z(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

Figure 1: Discriminator Objective Function

Figure adopted from this source. The discriminator tries to maximize loss, whereas the generator attempts to minimize it. In this objective function, $D(\mathbf{x})$ is the probability that, going back to the money analogy, real money is identified as real, whereas $G(\mathbf{z})$ is the synthetic data generated with respect to the noise $\mathbf{z}$.

$$\min_G V(G) = \mathbb{E}_{\mathbf{z} \sim p_z(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

Figure 2: Generator Objective Function

Figure adopted from this source. On the other hand, the generator attempts to minimize loss (we want to win by fooling the discriminator). Recall, G is the generator network that uses random noise z to make the synthetic sample and D is the discriminator that provides a probability from 0 to 1 on whether the input is real (we can assume 1 is real). Thus, GANs offer a feasible structure for generating synthetic data.

VAEs: I thoroughly enjoyed learning about VAEs because they employ a unique twist compared to GANs. Instead of generating data that is quite similar to the real training data, we can employ variability and optimize for a given direction when generating synthetic data. Variational Autoencoders, unlike traditional autoencoders (which don't enforce a latent space structure), are unique in that they enable a continuous latent space that permits convenient random sampling and interpolation. In short, a VAE consists of three main parts: 1) an encoder (produces parameters of a latent distribution) for converting the input data into a 2) probability distribution in a rich latent space and 3) the decoder for sampling from this latent space to generate variable data.

An interesting application of VAEs is conditional VAEs (CVAEs) that permit sampled outputs conditioned on specific encoded inputs. Researchers [19] have debated whether GANs or VAEs are better at producing synthetic data. In this paper, as literature has revealed that GANs are well suited for producing highly realistic samples (i.e., tend to enable higher accuracy in disease prediction, for instance [20]) and are popular implementations for synthetic data workflows, I experiment primarily with GANs, as well as autoencoders, and diffusion model.

4.1 Data Preprocessing

Before I dive into these various ML techniques for generating synthetic EHR data, I will outline three distinct notebooks I followed for preprocessing the MIMIC-IV dataset. These were notebook tutorials that I had found on GitHub and through publication references. In implementing these tutorials, I spent a large amount of my time understanding and replicating every code line-by-line and also contributing my own code for further analysis. The link to the notebooks is provided here.

4.1.1 Data Preprocessing Notebook 1

In this data preprocessing notebook, I learned about key steps for MIMIC-IV data preprocessing and classic ML tasks through scikit-learn (e.g., Random Forest for classification tasks). I first began by loading in the admissions, omr, labevents, d_labitems, diagnoses, d_icd_diagnoses, chartevents, and d_items files. Once done, I performed the following standard procedures: dropping duplicates, creating variable-specific datasets to analyze feature distributions to find outliers/missing values and perform imputations, fixing the dtypes (e.g., converting from object to numeric for variables such as result_value), computing summary statistics (count, mean, std, min, etc.), plotting distributions, sorting values to identify outliers, investigating patients who had multiple observations/fixing discrepancies with unit conversions (e.g., kg and lbs, 1kg is 2.2 lbs), using the median to impute for missing data, and merging tables to create one clean, standard dataset with all values (merging contingent on patient IDs). Once done, I implemented an ML mortality prediction task with numerical features and non-numerical features such as Age, Gender, Weight, CCU, CSRU, SICU, GCS_first, etc. using the scikit-learn library (e.g., X_train, y_train dataframes). Specifically, I trained a Lasso Logistic Regression Model, a technique for binary classification that implements L1 regularization with logistic regression, to classify whether a patient will survive or not (for this task specifically). This involved establishing numeric features (e.g., Age, Weight, GCS_first, Glucose_first, HR_first, ... Platelets_first, WBC_first, pH_first), defining non-numerical features (e.g., Gender, CCU, CSRU, SICU), and creating a classifier. For tuning the model to enable optimal performance, I implemented the GridSearchCV framework with $k=5$ folds (the model is randomly divided into 5 equal folds). When evaluated on the training set, the model had an average AUC of 0.7388. The odds ratio, which tells us about the relevance of features in the classification task, was also computed: WBC_first: 0.209885, Platelets_first: 0.636128, Weight: 0.690397, SICU: 0.691174, Glucose_first: 0.772380, HCT_first: 0.811725, PaCO2_first: 0.814652, K_first: 0.841636, HCO3_first: 0.860438, and Mg_first: 0.861533. An odds ratio of greater than 1 implies that the likelihood of death increases when the feature increases (i.e., there is a positive correlation), whereas an odds ratio of less than 1 implies otherwise (i.e., when feature increases, the likelihood of death should decrease. This is an inverse correlation). An odds ratio of 1 implies no effect on mortality prediction. The following confusion matrix and ROC curve were constructed with respect to the training data.

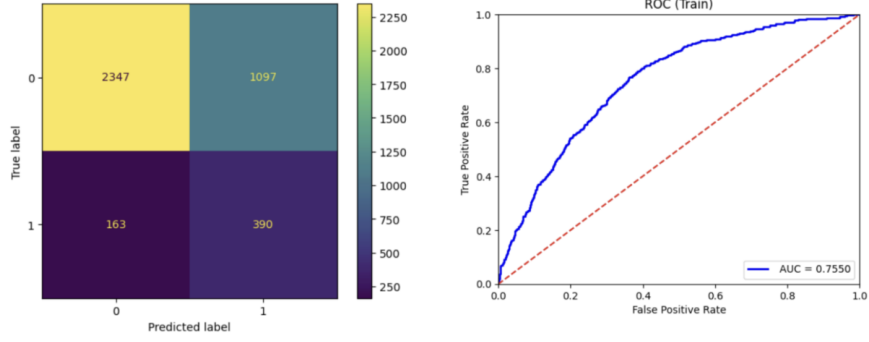


Figure 3: Correlation Matrix and ROC for Training

Ultimately the results were as follows. For classifying 0 (i.e., alive), the precision score was 0.935, recall was 0.681, and f1-score was 0.788. For classifying 1 (i.e., not alive), the precision score was 0.262, recall was 0.705, and f1-score was 0.382. There are 3444 support/true samples for “0” and 553 for “1” in the training data. While these results are not optimal, they are useful in understanding how the dataset may have been significantly imbalanced. From the confusion matrix, if we define “1” as a true case, there are 2347 true negatives, 1097 false positives, 163 false negatives, and 390 true positives. The AUC of 0.7550 illustrates that the model has a strong discriminative ability for classification, but is worth improvement.

When tested on the test dataset, the precision for “0” was 0.930, recall was 0.673, and the f1-score was 0.781 (support of 3427). For “1,” the precision was 0.261, recall was 0.695, and the f1-score was 0.379 (support of 568). Confusion matrix below and ROC curve below:

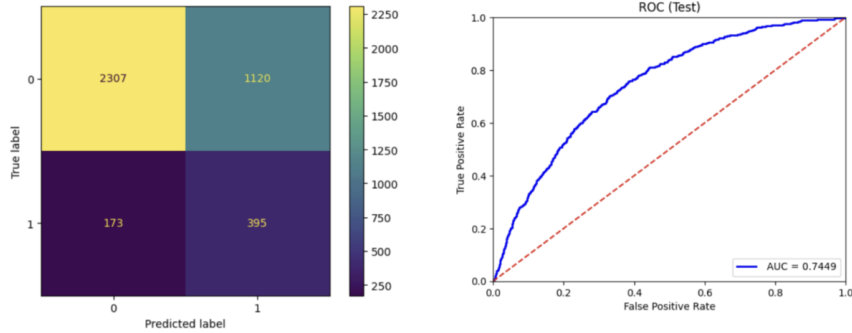


Figure 4: Correlation Matrix and ROC for Testing

From the confusion matrix, there are 2307 true negatives, 1120 false positives, 173 false negatives, and 395 true positives. Additionally, the AUC score is 0.7449, which implies that the model performance is roughly the same and similarly warrants improvement.

As a challenge to achieve higher accuracy and model performance, I implemented a Random Forest Model. The training results for the random forest model were as follows: for label “0,” the precision was 0.960, recall was 0.861, f1-score was 0.908 (support: 3444). For label “1,” the precision was 0.472, recall was 0.774, f1-score was 0.586 (support: 553). Below is a plot highlighting the most important features:

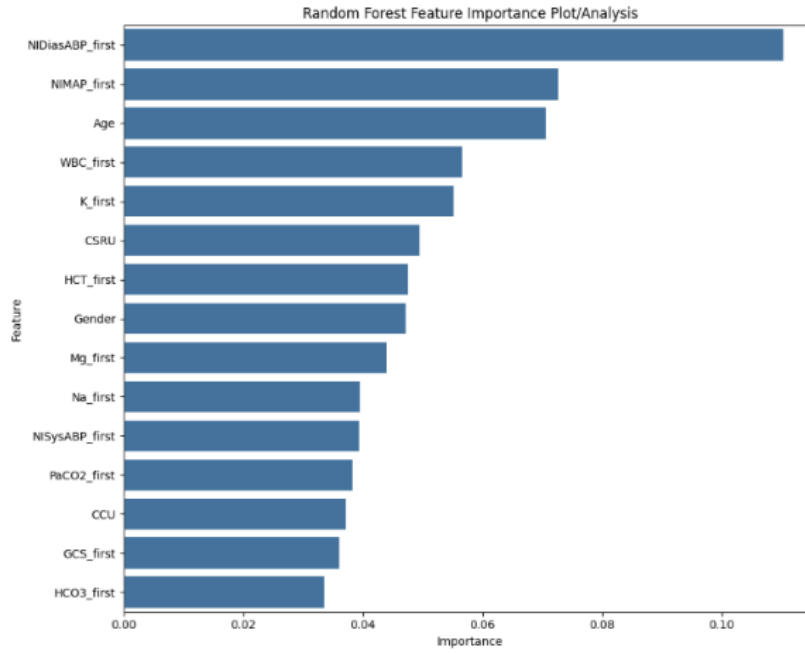


Figure 5: Random Forest Feature Importance Analysis

Ultimately, when comparing the Logistic Regression to the Random Forest model, the latter model performed better with a Training AUC of 0.9106 and a Test AUC of 0.7664.

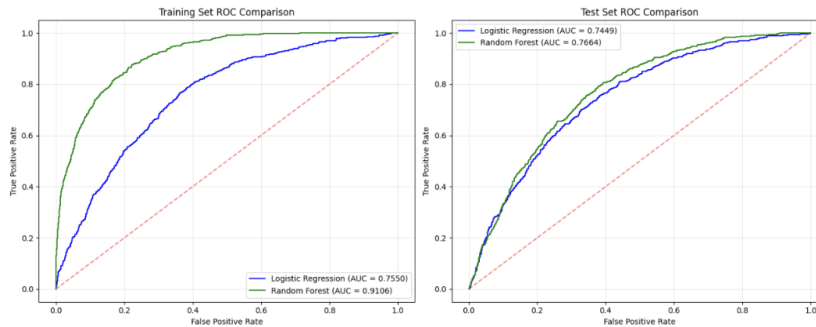


Figure 6: Training and Test ROC Comparison

For more plots pertinent to data processing, I have linked a document here (i.e., distribution plots, etc.).

4.1.2 Data Preprocessing Notebook 2

In this notebook, I performed similar data preprocessing techniques. That is, I loaded the admissions, patients, diagnoses.icd, and icustays dataframes, computed length of stay by handling the admittance/dischtime, and plotted several variable-specific distributions to assist with exploratory analyses, outlier detection, and dataset understanding. Several relevant plots (e.g., median LOS by diagnosis category, where the diagnoses were categorized into “skin,” “infectious,” “misc,” “genitourinary,” “neoplasms,” etc.) were constructed for understanding the data distributions. The ethnicities were also grouped into 5 main bins/categories: ASIAN, WHITE, HISPANIC/LATINO, BLACK/AFRICAN AMERICAN, and OTHER/UNKNOWN. NA values were replaced with 0, ICU data was merged with the main dataframe, and a categorized list of diagnoses were created for each admission (ultimately providing a merged admissions dataframe). Relevant plots from this notebook can be found linked here.

4.1.3 Data Preprocessing Notebook 3

For the GAN tutorial implemented in [3] in the methodology section, I had processed the data through the following steps: extracting patient cohort, ICD10 to ICD9 mapping, removing outliers, handling missing values through imputation, normalizing continuous columns, removing rare features (binary columns with less than 1 occurrence). The plots generated through this preprocessing notebook are provided here.

Overall, through these notebooks, I deepened my proficiency in data preprocessing techniques (removing outliers, performing data imputations, and more importantly, plotting distributions/understanding the data structure to find such outliers/missing values to begin with). I also engaged with Scikit’s library to perform and evaluate ML models with curated training and test data splits.

4.2 CTGAN (Conditional GAN for generating synthetic tabular data):

CTGAN developed by DataCebo, an AI startup focusing on synthetic data generation, enables researchers to generate synthetic data using the Synthetic Data Vault (SDV) Library.

To begin with CTGAN’s implementation, I first preprocessed the MIMIC-IV data by abiding by fundamental data preprocessing steps (removing extreme outliers, handling missing fields, imputations, etc.). Please see Data Preprocessing Notebook 2 for the detailed preprocessing steps followed. CTGAN focuses on generating synthetic tabular data through a neural-network based generator. CTGAN’s novelty lies in the invention of a “mode-specific normalization,” which handles the “non-Gaussian and multimodal distribution” and the design of a “conditional generator and training-by-sampling,” which handles the presence of “imbalanced discrete columns” in the dataset [8].

The authors use mode-specific normalization to represent the continuous values in the dataset, whereby each continuous column is treated independently of the other. Since real-world continuous data often consists of complex patterns (and isn’t necessarily nicely uniform), this technique seeks to enable the GAN to learn more meaningful patterns. Instead of simply squeezing the values between $[-1, 1]$ through min-max scaling, for example, mode-specific normalization enables the scalar value and the mode/peak within the continuous column to be one-hot encoded (i.e., per the paper, “each value is represented as a one-hot vector indicating the mode, and a scalar value indicating the value within the mode”) [8].

The second component of conditional generation seeks to address an issue similar to the “class imbalance” problem (which we can see is relevant from data preprocessing). The issue lies in the conventional method of training GANs, where the generator begins with some noise in the form of vectors sampled from a standard multivariable normal distribution. Then, once the generator and discriminator are trained together, the generator is able to map the standard multivariable normal distribution to the data’s real distribution. This method is not ideal as it does not account for the real imbalance in categorical columns. For instance, if we keep randomly sampling from the data during the training phase, nothing guarantees that underrepresented data will be seen enough. On the other hand, if we consistently resample rare samples from the training data, then we impose an incorrect distribution to be learned (we don’t want the GAN to think that rare categories are more common than they truly are). To address this limitation of existing GAN implementations, the paper seeks to sample evenly across all categories from discrete attributes through a conditional generator. The generated samples are produced given the values in particular discrete columns. In essence, the conditional generator is able to generate synthetic data rows upon being conditioned on a specific discrete column.

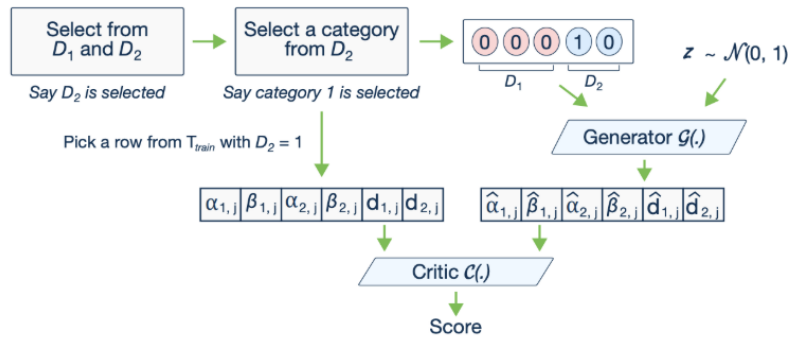


Figure 7: Conditional Generation Framework

The image above summarizes this conditional generation process where the category of 1 within D2 is selected (i.e., the synthetic data row is generated conditioned on this column).



Figure 8: Evaluation of Real vs Synthetic Data

After following the tutorial for CTGAN, I generated a dataset of synthetic patients. The folder for the synthetic data can be found [here](#). Once the synthetic data was generated, I then rigorously compared the synthetic data with the real training data. The following plots compare the distributions of the real and synthetic data with respect to various features/column variables such as the admission type, insurance, marital status, ICU, etc. Since these plots may be small in this report, I have linked a document for the individual plots [here](#). These plots were generated using the SDV library. Additionally, to compare the synthetic and real data, PCA was conducted. I have linked the code written to generate these plots [here](#) as `evaluatectgan.py`. The SDV library also offers the ability to conduct a quality report (i.e., evaluate the quality of the synthetic data with respect to the real data and metadata). The overall score/average for the quality report was 88.92%, indicating strong performance (the score is based on statistical similarity between the real and synthetic data distributions with a focus on the column shapes, column pair trends, and cardinality).

I have also computed the real correlation data matrix and synthetic data correlation matrix (plot available in document with individual plots); the mean absolute correlation difference between the real and synthetic data plots was 0.0719014100297452, implying a strong statistical similarity between the data distributions (i.e., we can observe similar correlations in both the real and synthetic dataset). Finally, a PCA test was conducted to enable dimensionality reduction (especially given that we have a lot of features) and understand how the distribution of the real and synthetic data compare to each other in this space. The similar data distribution supports the notion that we have generated reliable and high fidelity synthetic data.

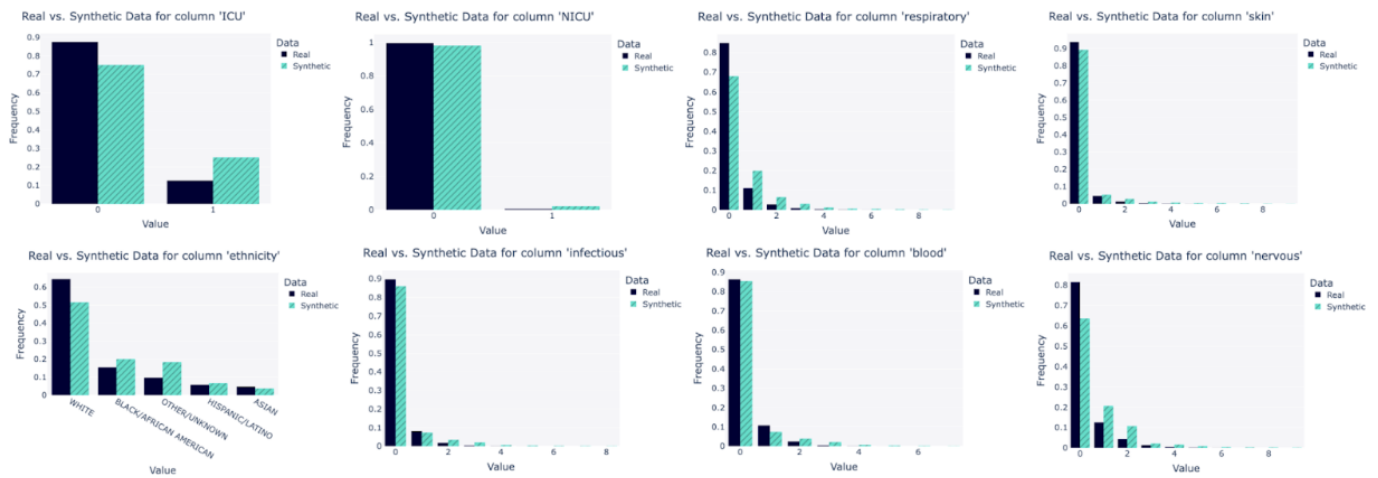


Figure 9: Evaluation of Real vs Synthetic Data Contd.

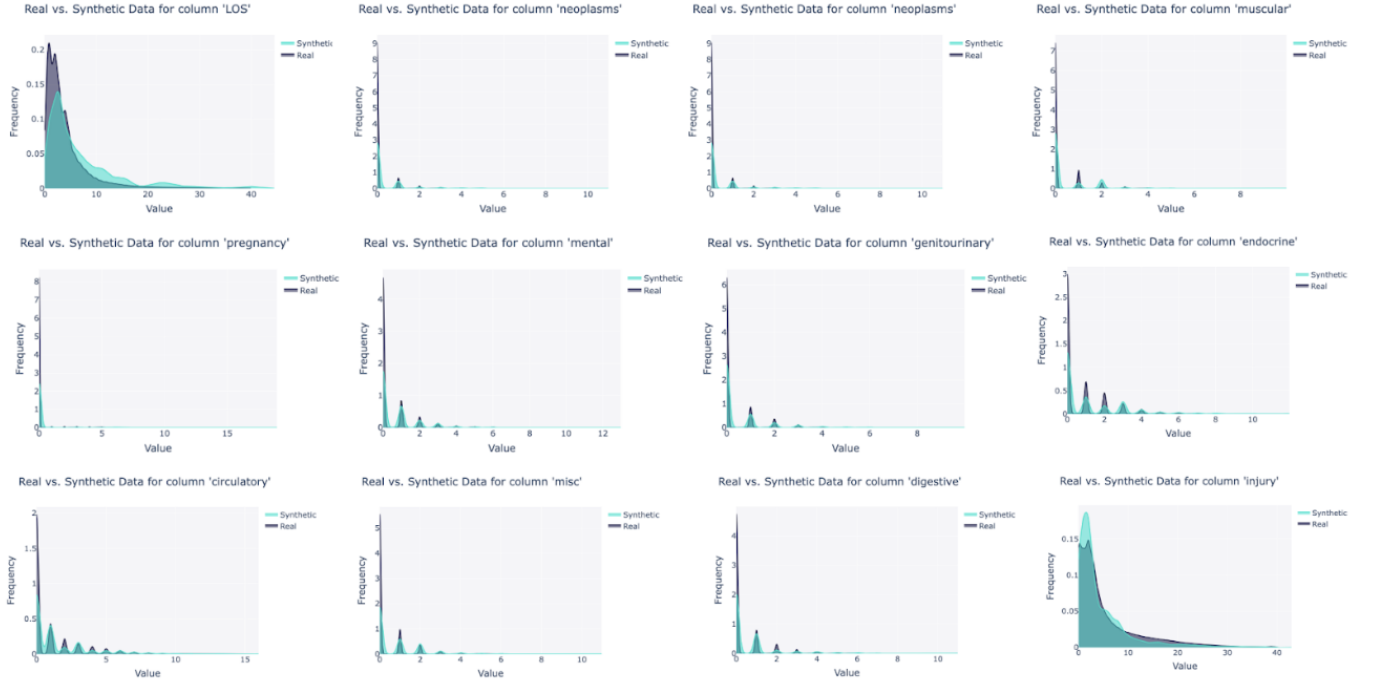


Figure 10: Evaluation of Real vs Synthetic Data Contd.

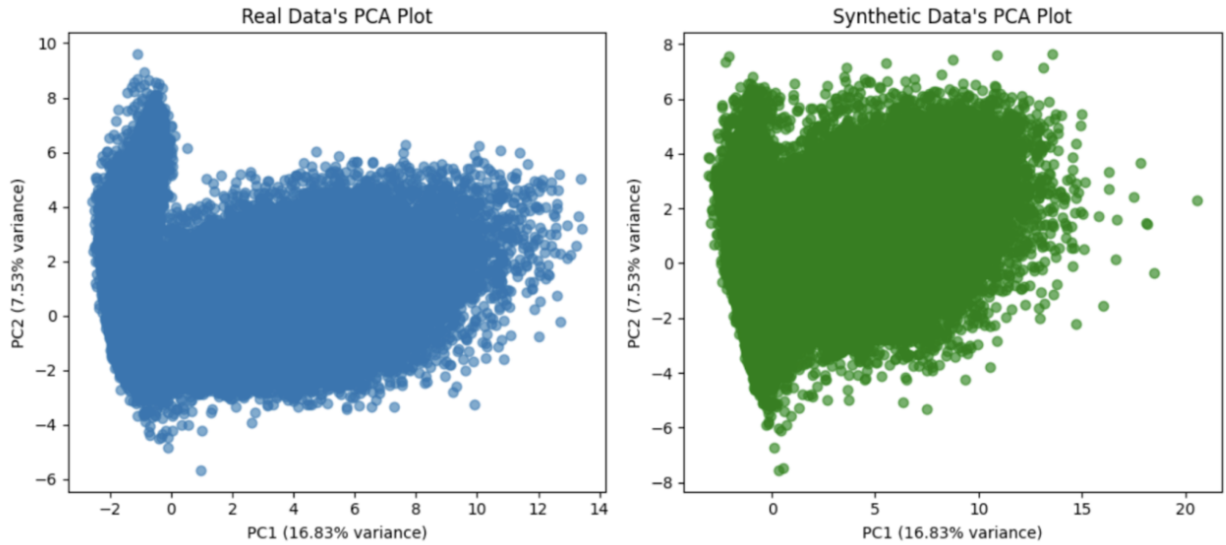


Figure 11: Real vs Synthetic PCA Plots

Overall, the data distribution bar charts and PCA plot indicate that the synthetic data quite strongly maps the distribution of the real data and is thus high fidelity synthetic data that we can use for downstream ML tasks.

Additionally, CTGAN was also used to generate synthetic patient samples from the EHRSHOT [21] dataset from Stanford's Shah Lab. The EHRSHOT dataset consists of 6,739 patients; 41, 661, 637 clinical events (rows in the ehrshot.csv file); 921, 499 visits; and 15 prediction tasks. First, since each row in the original EHRSHOT dataset represents a clinical event, I had prepared the data for CTGAN training by aggregating the features for each patient such that each row in the dataset would now represent a patient with a focus on the OMOP table types (e.g, measurement, observation, drug_exposure, etc.) The model was trained with 50 epochs to generate 10,000 synthetic samples (whereas the original dataset consisted of 6739 patients). An overall quality report score of 74.28% was produced, suggesting that although there is a strong statistical similarity between the distributions, the synthetic data's fidelity could be improved. I had randomly sampled 5,000 patients from both

the real and synthetic dataset when producing this metric. Code written to process EHRSHOT for CTGAN can be found here as [ctgancode.py](#).

The correlation matrix plot can be found in the same linked document. The data distribution comparison bar charts and the PCA plot can be found below:



Figure 12: Real vs Synthetic Data Distributions

Similarly, the data distribution plots for both the synthetic and real data indicate similar statistical distributions across the various discrete categorical data. Although the PCA data isn't as strong as for the MIMIC-IV data, I believe that this could be due to data sampling since we generated 10,000 synthetic patient samples from 6,739 real patients. Nonetheless, the PCA plots indicate similar directions and density for data point clustering, with the synthetic data being scattered a bit further out.

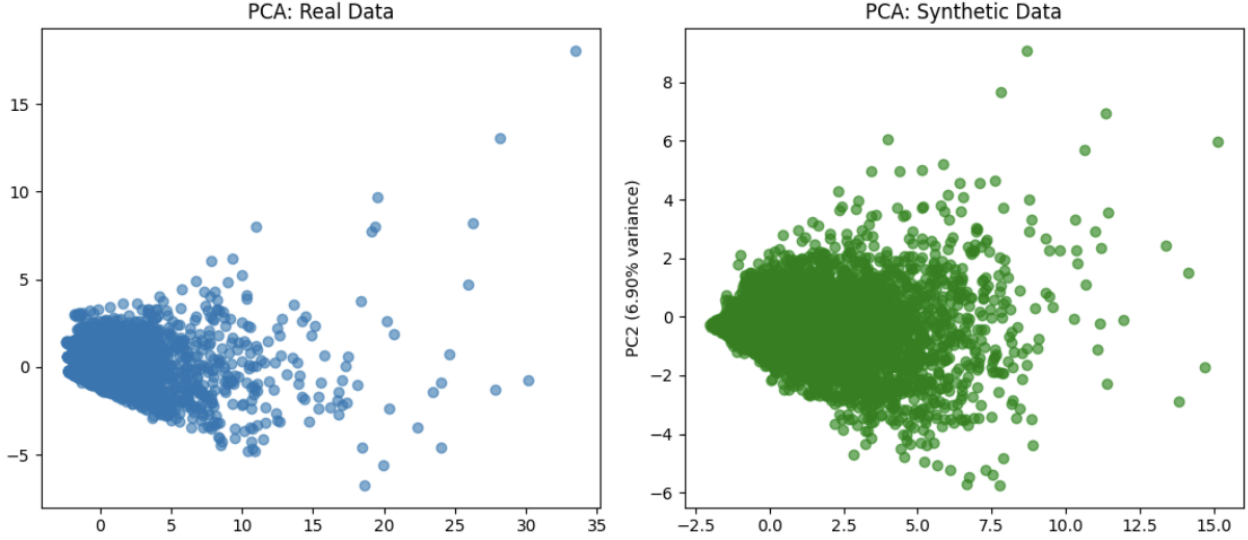


Figure 13: Real vs Synthetic Data PCA Plot

4.3 SynLongEHR

The SynLongEHR implementation was followed through the paper “On the evaluation of synthetic longitudinal electronic health records” by Jim Archterberg, Marcel Haas, and Marco Spruit (Leiden University Medical Center) [9].

SynLongEHR leverages two ML approaches, one of which is GAN-focused, (DoppelGANger [10] and CPAR) for synthetic data generation, while preserving the need for data fidelity, utility, and privacy. DoppelGANger specifically has been implemented for time-series data; it is a custom workflow that leverages the GAN architecture to enable “up to 43% better fidelity than baseline models” by solving challenges such as long-term dependencies and mode collapse [10]. On the other hand, CPAR [11] stands for conditional probabilistic auto-regressive model. I found this model to be suitable and appropriate to experiment with given that several of our research discussions prioritized conditional synthetic data generation (i.e., we want to model and distribute trends that mimic the real data in terms of conditional variances between certain variables, for instance). It is worth noting that, as discussed in the paper, the main goal here isn’t to generate the most optimal and high quality synthetic data possible, but rather to understand how GANs and conditional auto-regressive models handle generating synthetic longitudinal EHRs.

I implemented the tutorial aligning with this paper and generated the synthetic data ([link to folder here](#)).

The synthetic data consists of the following columns: subject_id, seq_num, age, gender, deceased, race, and icd_code.

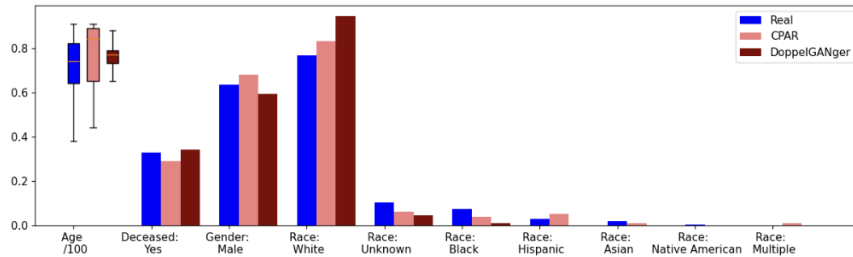


Figure 14: Real vs Synthetic (CPAR and DoppelGANger) Data

After generating synthetic data through both CPAR and DoppelGANger, the figure above was plotted to understand the distribution in terms of the first few features with respect to the varying values in each category. As shown, the distribution between the real and synthetic datasets is relatively preserved. This is expected and ideal as, although the synthetic data is meant to be different from the real data, its distribution is meant to mimic that of the real data (to preserve relevant trends for downstream ML prediction tasks, for example). Additionally, the following heatmap plots were produced to highlight the disease progression patterns across all

patients combined (in comparing the real MIMIC-IV dataset, the synthetic data from CPAR, and the synthetic data from DoppelGANger).

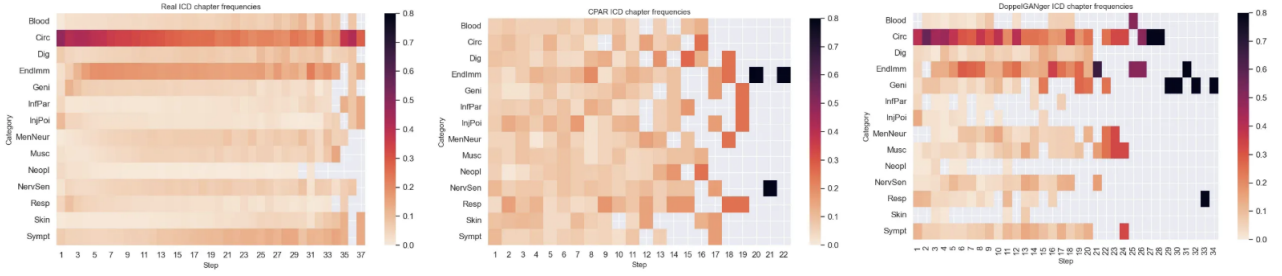


Figure 15: ICD Heatmaps - Real vs Synthetic

In reviewing these plots, it seems as though CPAR (middle) had overall performed more optimally in generating the synthetic EHR data. This is supported by the natural progression of ICD diseases over each step (i.e., CPAR has a more gradual progression that aligns with the real dataset). In these plots, the x-axis is the time steps in the patient sequence and the y-axis is the various ICD-10 (international classification of diseases) chapter categories. The frequency or probability of each ICD disease/condition occurring at each time step is shown by the heatmap scale (from 0 to 0.8), where the more intense the color, the higher the probability.

The real dataset (on the very left) demonstrates a smooth and gradual pattern over time with circulatory conditions having a high probability throughout the patient steps. Overall, most of the ICD categories demonstrate stable patterns across the patient sequence (i.e., there aren't abrupt changes as we may see with the other plots, notably DoppelGANger).

CPAR (in the middle) illustrates a bit more of a sparse/scattered and choppy disease progression. In time-steps 1-18, it seems as though there is a natural progression for most of the ICD categories. Though, it is evident that certain categories such as NervSen and Endlmm demonstrate high-probability spots that are rather abrupt (and towards the end of the steps). In essence, CPAR appears to have extracted the overall temporal dependencies effectively with less smoothness/gradualness towards the end of the patient steps.

Lastly, the DoppelGANger data (on the very right) is arguably less consistent/gradual, yet appears to infer the density for the circulatory category. The data appears more fragmented, noisy, and scattered. There are several high-probability spots that appear rather abruptly towards the end of the patient steps (for instance, for Resp, Geni, and Endlmm). Thus, overall, there is a less coherent temporal progression through the patient distribution when placed in comparison to the real MIMIC-IV patient dataset. In comparing CPAR and DoppelGANger, CPAR appears to be more effective in capturing more smooth/gradual temporal dependencies for ICD categories.

4.4 Generating Synthetic Electronic Health Record Data Using Generative Adversarial Networks: Tutorial

In our most recent meeting on March 13th, we discussed the feasibility of generative adversarial networks (GANs) as a reliable and powerful ML architecture to generate synthetic data. Following this discussion, I had begun to self-learn about GANs through Ian Goodfellow's original paper on GANs [22], which I am still yet to fully comprehend, and his introductions/video tutorials from NIPS 2016. In learning about GANs, I also found the linked YouTube video by DeepBean particularly helpful (brief learnings previously documented in report).

The GAN trained and used to generate synthetic data for this section was obtained from the following paper: Generating Synthetic Electronic Health Record Data Using Generative Adversarial Networks: Tutorial by Chao Yan, Ziqi Zhang, Steve Nyemba, and Zhuohang Li, researchers at Vanderbilt University [7].

I have provided a folder with trained model checkpoints and the synthetic data generated here. This tutorial implemented the MIMIC-IV dataset. In summary, I leveraged the GitHub repo provided alongside this paper and preprocessed the data, trained the GAN model, and then used the best model checkpoint to generate the synthetic dataset. The overall reproducibility workflow I performed was as follows: 1) Data preprocessing steps (i.e., normalizing continuous variables, handling missing values, removing outliers, etc.), 2) training the GAN model (2x NVIDIA RTX 6000 Ada Generation), 3) using the best trained model checkpoint (at 5900 epochs) to generate the synthetic dataset, and finally, conducting data evaluation measures to understand synthetic data quality, which I conducted through a script I wrote.

As I was training the model, a somewhat counterintuitive point that challenged me was understanding how the highest quality synthetic data isn't necessarily produced from the model checkpoint with the lowest

training loss [7]. In fact, overtraining the GAN model can reduce the quality of the synthetic data generated (overfitting is one prime consequence, for instance). The steps taken for preprocessing the data can be found in Data Preprocessing Notebook 3 described above. I have placed a larger focus on discussing the results in this section.

The shape of the real data (preprocessed training data) is (171725, 1468), with the first few column names being “WHITE,” “BLACK,” “ASIAN,” “HISPANIC,” “UN,” “OTHER,” “DIE_1y,” “GENDER,” “008,” etc. (i.e., other ICD codes). Besides these ICD codes, towards the end of the datasets, there are also the following labels: “AGE,” “BMI,” “DIASTOLIC,” and “SYSTOLIC”. The shape of the synthetic data is as follows: (515212, 1469) with a total of 1468 features and the first column being the “sample_id,” which isn’t a relevant feature here.

In order to compare real and synthetic data rigorously, I have written a comprehensive script in Python linked here as `evaluate.py`. I had written this script to deepen my understanding of data evaluation measures and to rigorously test the data produced for quality, privacy, etc. I have also provided comprehensive comments in this script detailing my thought process as I was learning the various techniques of data assessment/evaluation. The script consists of the following functions: 1) `statistics_comparison`, 2) `distribution_comparisons`, 3) `correlation_analysis`, 4) `machine_learning_utility`, 5) `dimensionality_reduction`, and 6) `privacy_assessment`. The real data consists of 171, 725 patients with 1468 features (thus, a very high dimensional dataset). The synthetic data consists of 515, 212 samples with 1468 features (let’s exclude the patient ID). For 1), I computed the mean, std, min/max, and skewness for the first 10 features. For example, for the feature `DIE_1y` and `GENDER` the `Mean_Diff`, `Std_Diff`, and `Skew_Diff`, respectively were {0.36622, 21662, 51104} and {0.12493, 0.38415, 0.36869}. For 2), I compared the distribution of the real and synthetic data for the first 25 features through both the Kolmogorov-Smirnov test and Mann-Whitney U test. The lack of strong similarity across these distributions revealed that the synthetic data did not strongly represent the distribution of the real data. Nonetheless, I proceeded with further analyses. For 3), I studied how effective the synthetic data is at preserving the relationships between various features by computing the correlation matrices. First, I computed the correlation matrix for the real dataset, then the synthetic dataset, and then finally, a meta-correlation between the real and synthetic correlation.

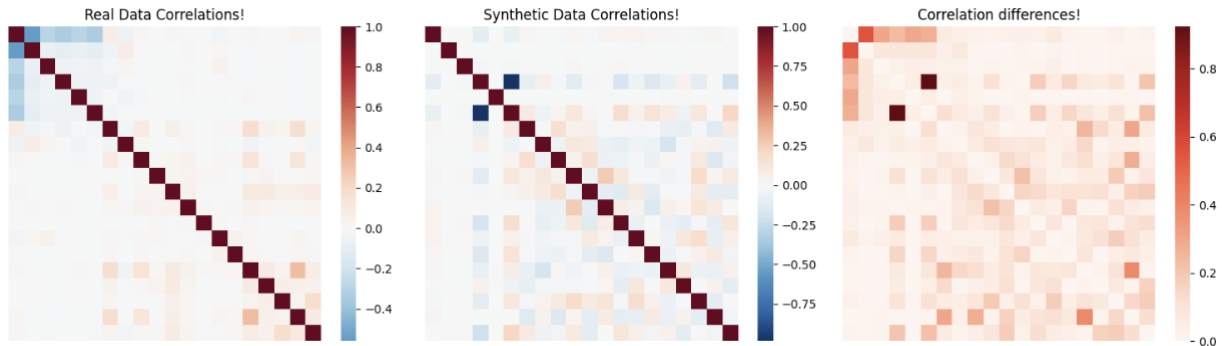


Figure 16: Correlation Analysis for Real vs Synthetic

The plots of these correlation matrices are shown above. We can see that for the final plot, the correlation differences aren’t radically different, which suggests that the generated data is presumably strong, but not optimal (the lack of correlation differences suggests that the synthetic data mimics the real data well, but we also have to consider the results from 2 as well). For 4), I conducted a standard ML prediction task for predicting the output variable `DIE_1y` with both the synthetic and real data as training data (the real data as test data in both cases). I utilized Random Forest since this had performed well in the data preprocessing stages and separately, is a robust algorithm.

For 5), I proceeded with dimensionality reduction techniques to compare the real and synthetic data since the data was very high dimensional with 1468 features. I conducted an analysis for both UMAP and PCA. The plots for these are shown below:

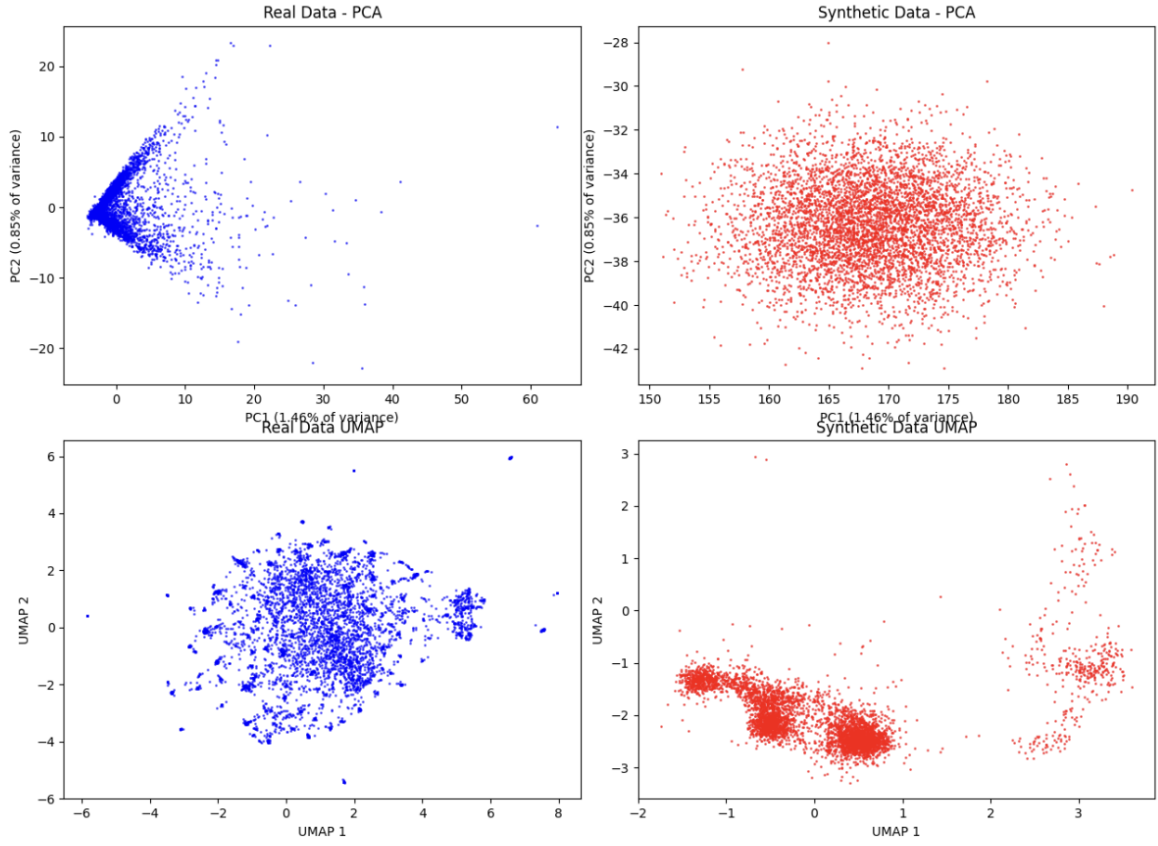


Figure 17: PCA Plot for Real vs Synthetic Data

Ultimately, since the distributions/scattered data points are quite different for both the real and synthetic data, it is evident that the model training can be improved. In future iterations, it would be productive to further test out several model checkpoints, conduct hyperparameter optimization (i.e., play with epochs, learning rate, etc.) to identify which checkpoint would support the most ideal synthetic data generation (i.e., similar data distributions). It is worth noting here that the model checkpoint with the least loss won't necessarily provide the best data.

Finally, for 6), I attempted to learn more about privacy assessment by computing the euclidean distance between each synthetic patient and the nearest real patient. In doing so and computing the mean/std distance, as well as establishing a privacy threshold (cut-off defined at bottom 10%), I was able to obtain the following figure, which suggests that most of the data is safe (i.e., the small fraction on the left of the risk threshold is concerning).

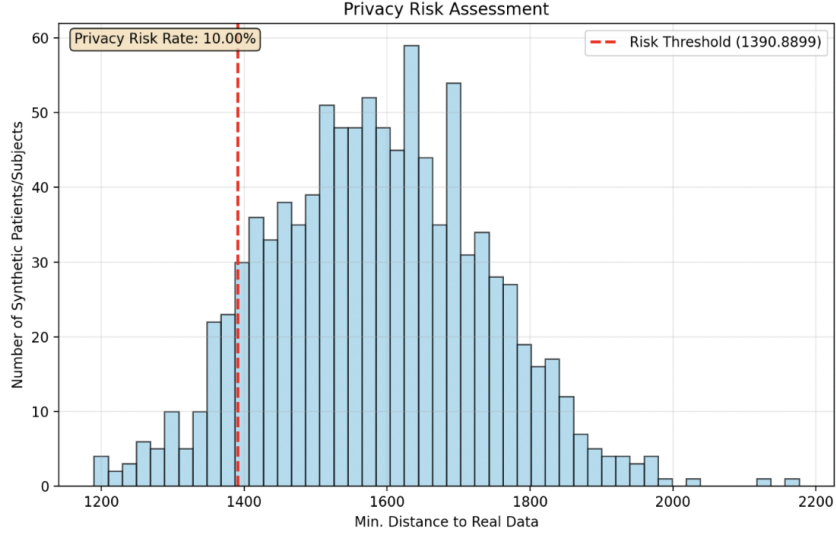


Figure 18: Privacy Risk Assessment Plot

The mean minimum distance to the real data was computed to be 1585.7010 (std. for minimum distance is 149.8749). Overall, this implies that there is a diverse distribution of unique patients (when comparing the real and synthetic data), and proposes a unique trade-off between similarity and privacy.

Ultimately, I enjoyed writing this script as ultimately, it felt cohesive in tying my knowledge from data preprocessing and ML to assess the data, as well as provided greater insight on the privacy and fidelity of the synthetic data generated.

4.5 MedGAN

In the paper “Generating Multi-label Discrete Patient Records using Generative Adversarial Networks,” [12] the authors propose a novel GAN architecture called medGAN that is known to generate high-dimensional synthetic data through both an autoencoder and a GAN. I found this to be a novel, interesting approach to experiment with since the researchers employ the usage of both the autoencoder and GAN architectures, which I discussed are feasible and reputable for synthetic data generation. More importantly, the code supports patient data over time (i.e., in a time-series fashion), which is important to our understanding of how patient trajectories are formed (e.g., how EHR data looks over time). It is important to note why the autoencoder is leveraged in this architecture; the paper reveals how through backpropagation, the generator G is trained by the error signal produced by the discriminator D . During this process, the GAN’s limitation lies in only being able to “approximate discrete patient records... with continuous values.” To counter this, the autoencoder’s ability to let high-dimensional data be distributed into a latent space that can be sampled from to decode new data enables it to identify and learn salient features of the discrete variables in this latent space. The decoder can decode such salient features into discrete records. [Link to repo.](#)

MedGAN uses the sigmoid function to generate the output. That is, although the training data was formed through a binary output matrix file, the synthetic data produces probabilities from 0 to 1 for each feature. We can accordingly threshold these values (e.g., standard threshold would be 0.5) to assign a 0 or 1 label to each computed probability.

In the initial training run, the results were as follows:

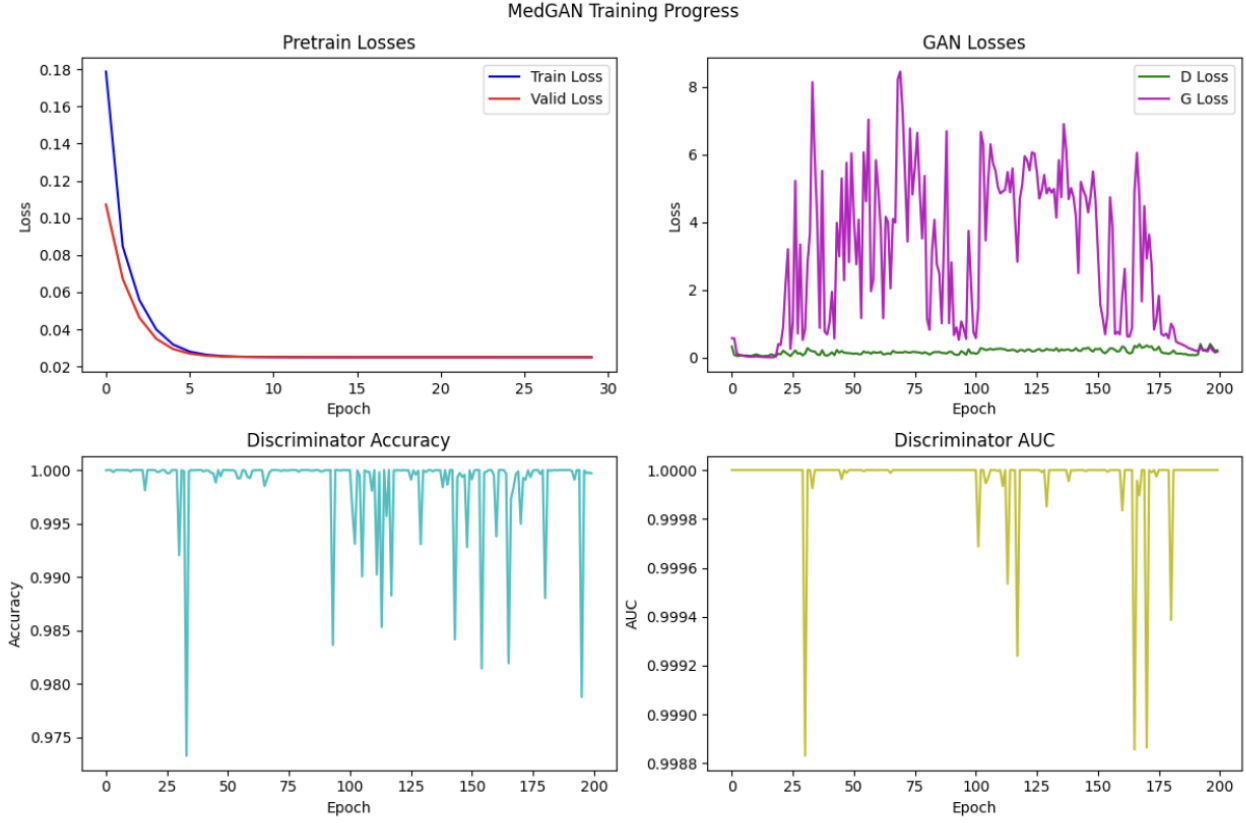


Figure 19: Training Results from Initial Run (Mode Collapse)

Based upon these results, it seemed as though the generator was not learning to “trick” the discriminator significantly/effectively. That is, there was a mode collapse. Thus, for this reason, I had reduced the discriminator learning rate (i.e., prevent the discriminator from learning too quickly), adjusted the training frequencies, added noise to the discriminator, and employed label smoothing as steps to improve the GAN performance. Specifically, the generator would now train twice per batch (instead of once) and the discriminator would now train once per batch (instead of twice) and a 0.9 label smoothing was applied to the discriminator’s training labels to enable more steep gradients that could be helpful for the generator to learn more information. Label smoothing prevents the discriminator from becoming too confident. By adding noise to the discriminator, we also prevent it from overruling the generator (i.e., we can impose noise through dropout, e.g., from 0.5 to 0.7). The higher dropout rate of 0.7 used was more effective since the dropout of 0.5 enabled 99% accuracy for the discriminator, whereas the 0.7 dropout enabled 95% percent accuracy (with some dips), permitting the generator to learn rich information. The learning rate for the discriminator was also reduced from 0.001 to 0.0001 to prevent the discriminator from learning too quickly such that the generator could not learn any meaningful trends for generating the synthetic data. The plot below was produced after making these changes:

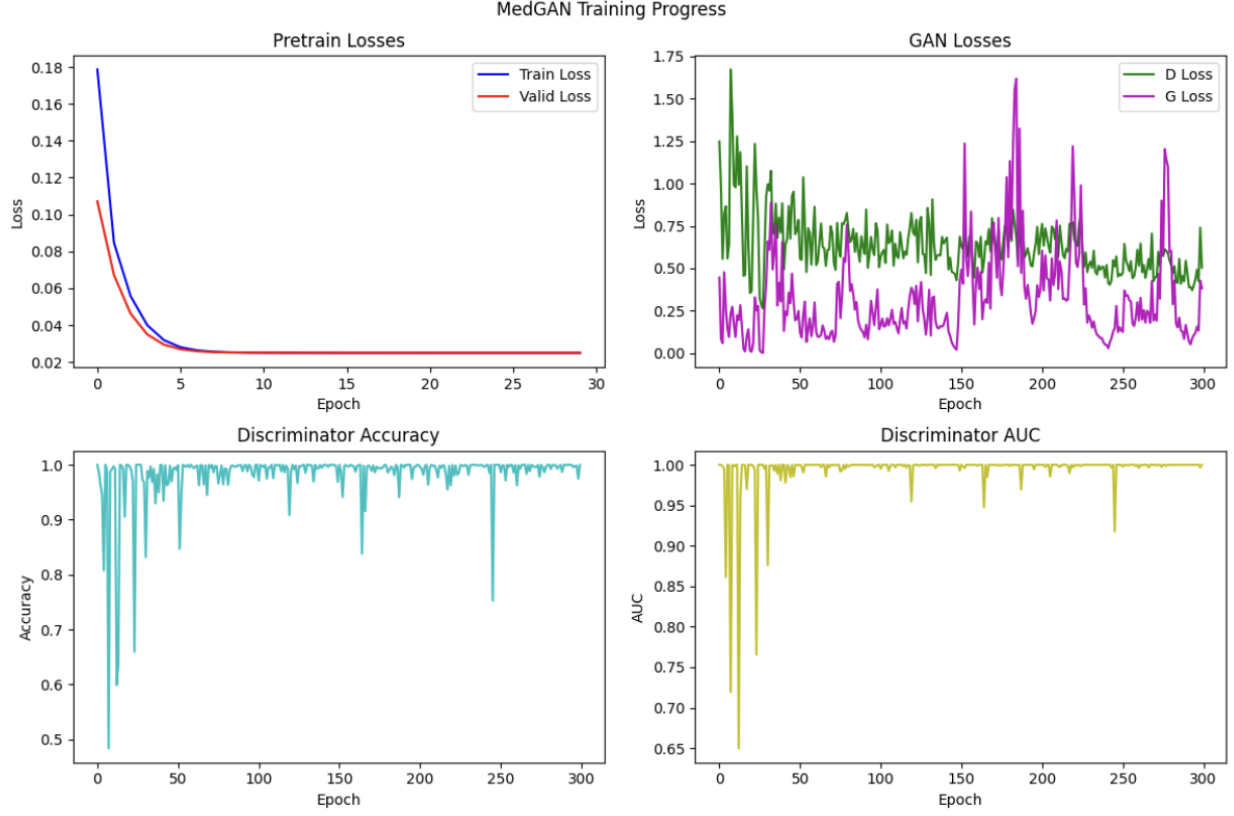


Figure 20: Improved GAN Training Results

These results are much better as there is a more balanced GAN adversarial training (we can see the D and G competing). In the previous run, the discriminator was completely dominating. Additionally, we can see that the discriminator accuracy drops to a record 0.65, which is more ideal as this demonstrates that these models are learning over time (as opposed to the discriminator dominating from the very beginning). The generator is challenging the discriminator, which enables us to get a trained model that will generate meaningful synthetic data. The synthetic data generated from this implementation can be found [here](#). Note that the initial code implementation on GitHub doesn't demonstrate these plots; I had produced these plots to improve training and data generation quality.

4.6 ScoEHR

Finally, in the paper ScoEHR [13], the authors similarly employ the autoencoder in conjunction with a deep learning diffusion model (as opposed to a GAN). I found this particularly interesting, especially since continuous-time diffusion models are utilized, which I did not come across in my research before. Additionally, ScoEHR is known to outperform the methodology implemented in the previous paper medGAN. Because I could not gain access to the dataset required from the Yale New Haven Health System as I could not find this publicly available, I trained the ScoEHR model through MIMIC-III (with modified training config and processing/data preparation).

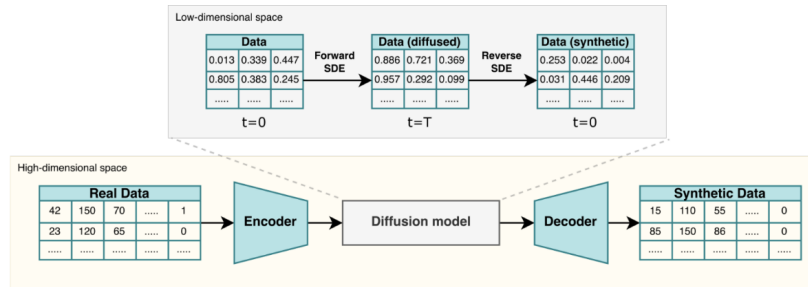


Figure 21: ScoEHR Architecture

Figure above adopted from paper [13]. For ScoEHR, the EHR data from MIMIC-III is converted into a low-dimensional latent space through a pre-trained autoencoder. In this latent space, the data is perturbed over the time interval $[0, T]$ through a variance preserving SDE. In order to then generate novel samples from this sample space, a reverse-time SDE is utilized. I found this paper to be highly relevant to building off of the work in the previous medGAN paper as it uses a novel twist that doesn't require a GAN's adversarial training (and thus allows for more stable training as the paper argues). Ultimately, the authors show that ScoEHR outperforms medGAN for metrics such as log-cluster, SRA, and PCD. ScoEHR took approx. 12.5 hours to train on my local machine. The synthetic data generated for ScoEHR can be found here.

The real data consisted of 46520 samples and 1071 features; the synthetic data produced consisted of 1024 samples with 1071 features (generated in 32 batches, so 32 samples per batch). In order to assess the data quality, I leveraged the same evaluation script I wrote in the GAN tutorial (3) with minor modifications to load the .npz synthetic data file and the .matrix real training data file and appropriately process the data.

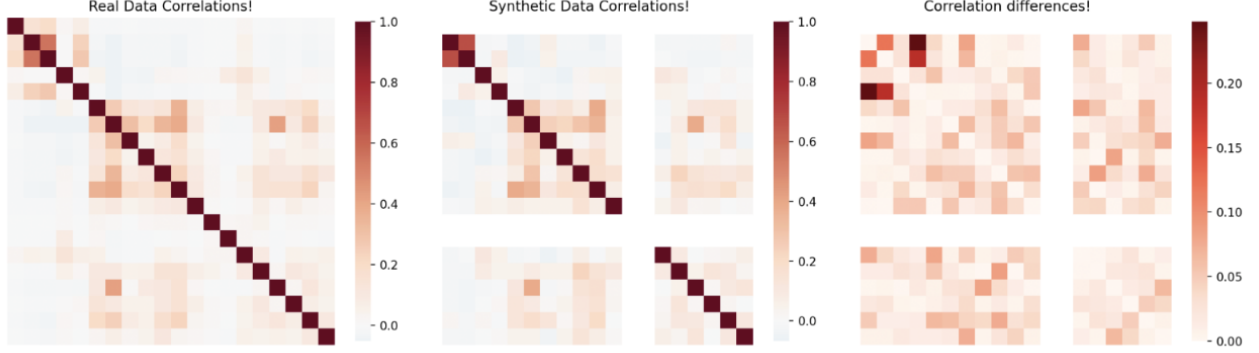


Figure 22: Correlation Analysis for Real vs Synthetic

The right-most correlation difference plot implies that most, if not all, correlation differences are below $\tilde{0.15}$. In methodology [3] (GAN tutorial), most points were above or close to 0.20. The presence of a smaller correlation difference implies that the synthetic data and real data are correlated in similar ways (i.e., as expected of high fidelity synthetic data).

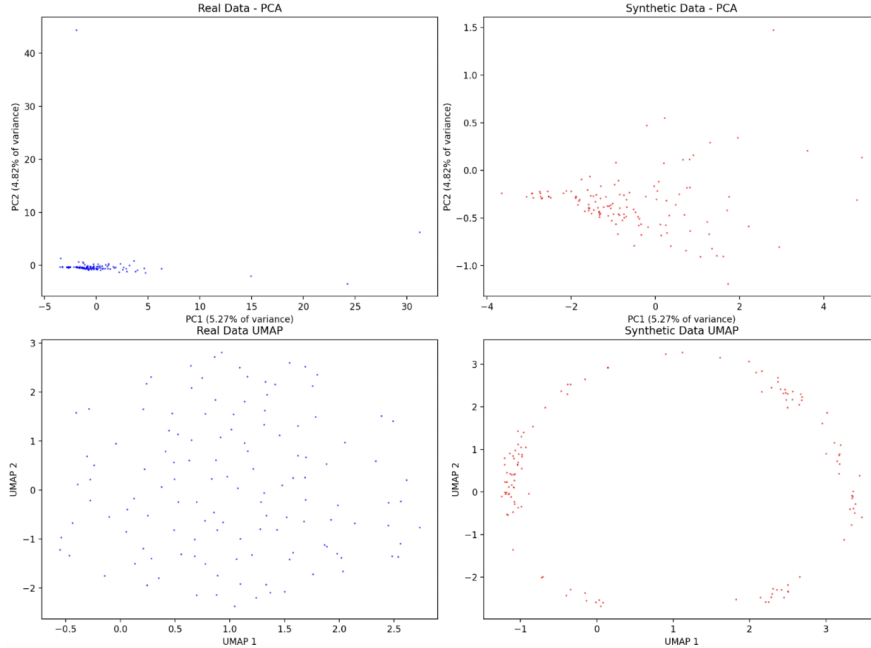


Figure 23: PCA and UMAP Plot for Real vs Synthetic

The PCA plots above could imply that the synthetic data is imposing some hidden relationships or capturing simply arbitrary relationships. The synthetic data for the PCA appears more spread out; for the UMAP, it is also spread out, but with no concentration in the center.

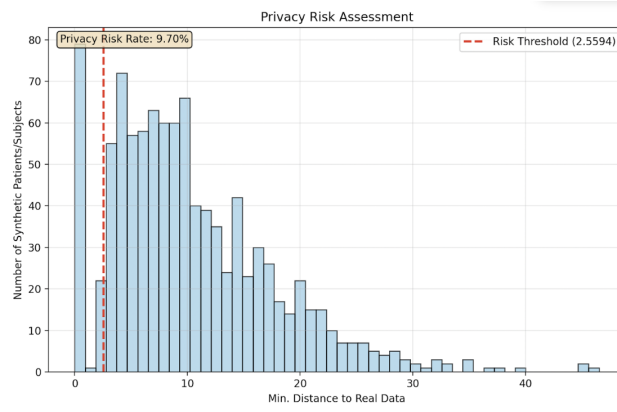


Figure 24: Privacy Risk Assessment

As a part of the same evaluation script written for the GAN in (3), I conducted a privacy risk assessment, since this was a key idea we discussed regarding the motive for generating synthetic patient data (see literature review section at end). The mean minimum distance to the real data is 10.2772 and the standard deviation for the minimum distance is 7.3344. This introduces a unique trade-off, where the synthetic data may mimic the real data well at the expense of potentially introducing privacy issues.

4.6.1 EHR Dashboard Prototype

Below is the final interface prototype that I have worked on for understanding and visualizing EHR trends over time. I have retrieved this prototype and elaborated on it below. I have provided screenshots of the interface below and briefly described the features of each page. The prototype I developed consists of 7 tabs: Timeline/Trends, Clinical Analysis, Alerts, Notes Analysis, MedQuery, Precision Cohorts (inspired by

's paper "Precision population analytics: population management at the point-of-care" [23]), and the Raw Data in json format.



Figure 25: Landing Page

The landing page enables the doctor to pick the relevant patient ID and offers the information about the patient's timeline/trends over time, as well as personal information such as the Age and Gender.



Figure 26: Laboratory Results

Further down on the same page, the laboratory results (specifically, the heart rate, diastolic BP, and systolic BP changes can be viewed over time). There is also a symptom frequency over time chart that illuminates the frequency of the reported symptom for all of the visits in a given month (e.g., a high symptom frequency of dark red for “shortness of breath” would mean that in all of the visits/measurements for that corresponding month, the patient has reported a shortness of breath on nearly all visits. This would be a persistent symptom present).

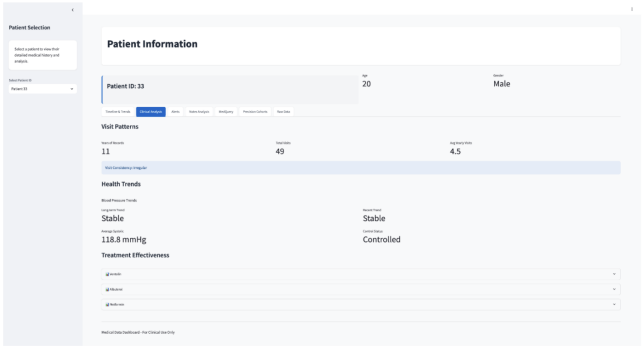


Figure 27: Clinical Analysis Tab

The Clinical Analysis tab provides data on the years of records, number of total visits, average yearly visits, as well as general health trends. The effectiveness of prescribed medicines can also be inferred (e.g., for how many visits was the medication recommended, treatment periods, average changes in vital signs that correspond to the medication usage).

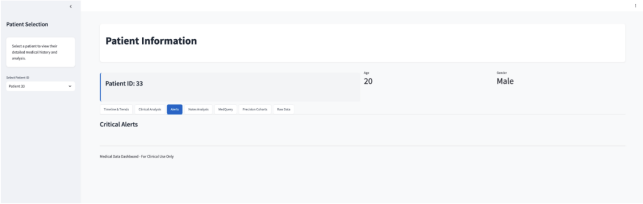


Figure 28: Alerts Tab

The Alerts tab is intended to flag any drastic or irregular changes in vital signs/the patient’s overall health profile. At the moment, there are no alerts present here.

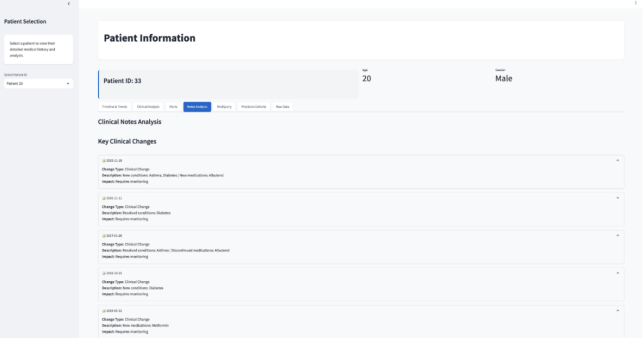


Figure 29: Notes Analysis Tab

The Notes Analysis tab is meant to provide a brief summary of past visit-based notes (i.e., what was the description of symptoms, what medicine was administered, what were the protocols or next steps to follow, etc.)

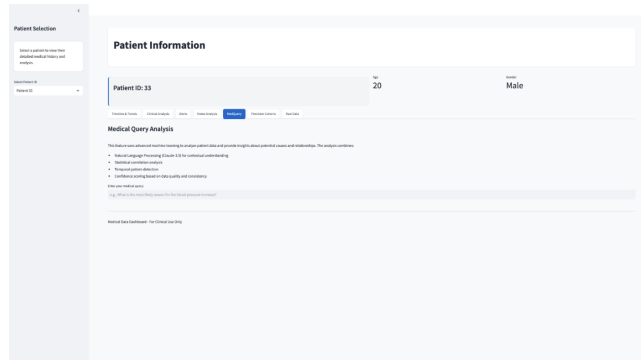


Figure 30: MedQuery Tab - LLM Prompting

The MedQuery tab leverages Claude’s LLMs for contextual understanding and questions the doctor may have. This LLM model was not fine-tuned and was implemented through an API for simply demonstration purposes.

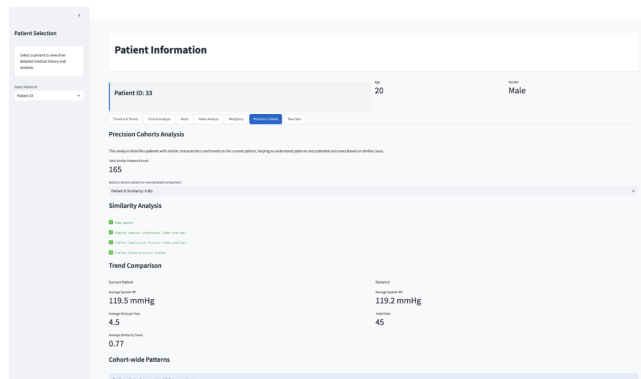


Figure 31: Precision Cohorts Analysis Tab

The Precision Cohorts Analysis tab offers a direct comparison to similar patients based on factors such as gender, medical conditions, medication history, and blood pressure trends.

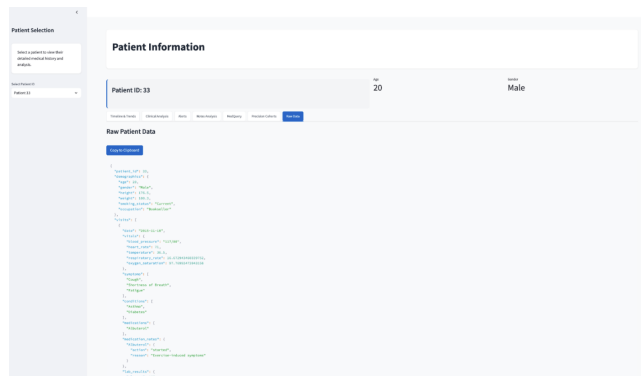


Figure 32: Raw Data Tab

Finally, the raw data section clearly depicts the data recorded at each visit for the patient (features include age, gender, height, weight, smoking status, occupation, date, vital signs such as blood pressure and heart rate, symptoms, conditions, medications, notes, etc.)

In developing this prototype, I challenged myself to learn more about the front-end stack (CSS, React, JavaScript) and after developing a basic template for these different tabs, I leveraged Replit and Claude for debugging and further learning about web-development. It is worth noting that the prototype was primarily developed for demonstration purposes. Thus, for the sake of the interface demonstration, the patient data generated did not use any deep learning architecture, but was rather conditionally coded in a very trivial fashion. That is, whereas variables such as age and gender were random, an effort was made to map conditions to typical symptoms and medications (e.g., for Diabetes, potential symptoms could be fatigue and nausea,

medications could be Metformin, and a potential reason for starting the medication could be high blood sugar levels). Additionally, seasonal symptoms were added (i.e., in the Spring, a cough was added with 70% chance, in the Summer, fatigue was added with 60% chance, and in the Fall, symptoms such as cough and shortness of breath were added with an 80% chance). Similarly, systolic, diastolic, and heart_rate values were generated based upon other conditions (e.g., if the patient has hypertension, the base systolic value is 138, else 120, with these values being supported from Google searches). Thus, although this was a less robust implementation for synthetic data (without an ML architecture), the intent was to generate reasonable/qualifying patient data to understand how the interface could interact with such data. Through my previous work in the methodology section (exploring GANs, autoencoders, and diffusion models), I have provided more reliable, robust, and high fidelity synthetic datasets to use. I have provided the code for the interface data generation here. Throughout the quarter, I also experimented with several implementations and prototypes local to my machine (e.g., localhost), which are not depicted here for conciseness, as the prototype shown above was my latest prototype.

5 Limitations

While employing powerful deep learning architectures such as GANs, VAEs/autoencoders, and diffusion models has led to generating synthetic data that is promising for downstream ML prediction tasks and other experiments with LLM-information retrieval (through RAG, prompt engineering, etc.), it is important to highlight a few limitations. 1) In order to deploy these model architectures so that they are useful to institutions/facilities, we need to ensure that we can meaningfully process EHR data with ease. Due to the variability in how EHR data can be stored and processed from one facility to another (e.g., dealing with multiple modalities and processing such data is one example), we need to ensure we can develop a standard for data standardization/processing or at the very least, provide very clear documentation, 2) for conducting downstream tasks that pertain to LLM-information retrieval through techniques such as prompt engineering, RAG, etc., it is important to highlight how LLMs aren't always correct. To this date, researchers are still attempting to understand interpretability in LLMs and why they may hallucinate, with a recent paper from OpenAI revealing that we should reconsider our evaluation incentives to minimize hallucinations [24]. Nonetheless, for medical decisions, we need to ensure we do not establish an overdependence on LLM outputs. 3) Data sparsity and structure is another key limitation. In searching for various datasets, I found MIMIC-IV to be the most structured and compatible for my research. More often than not, other datasets were highly unstructured, not easy to use (not standardized, a lot of missing values, etc.), and subject to very lengthy training. While data sparsity may not be a pressing limitation for hospitals that have direct access to confidential patient data, the need to structure such data in the appropriate training format is vital (if we choose to generate synthetic data for ML tasks or simply use the real data), 4) broadly, there may be ethical and legal concerns about generating synthetic data that may risk the privacy and confidentiality of patient's healthcare information. We need to continue to ensure that we implement robust measures that can't enable people to decode generated data to the real training data (or identify leakages), 5) finally, GANs may risk issues such as mode collapse (for VAEs, posterior collapse could be a risk). Thus, while these model architectures have been very promising, we need to make active efforts to explore diffusion-based models (or other architectures), for instance, to expand the possibilities for synthetic data generation.

Additionally, more specific to the results of this paper, a few limitations are the need for reliable access to compute, the publicly available datasets (i.e., these are prepared by medical institutions, were mostly ICU with the exception of MIMIC, and a lot of patient information is also not shared), the need to incorporate multiple data modalities (images, video, audio, natural language, etc.) to allow for more rich synthetic data generation and analyses of how LLMs interact with such data. Furthermore, a lot of deep learning workflows from online papers did not have their code publicly available for reproducibility and experimentation.

6 Future Work

Thus far in my research experience, I have 1) conducted an extensive literature review on EHRs and how LLMs can be applied in medical information-retrieval contexts, 2) self-learned various important topics such as RAG, prompt engineering, fine-tuning, etc. with a few implementations as discussed in this paper, 3) written, implemented, and extensively debugged Python code for generating and evaluating synthetic patient data, 4) gained a deeper understanding of GANs, VAEs/autoencoders, and diffusion models for domains such as data generation and model performance (e.g., training and evaluating models, adding noise, label smoothing, etc.), and 5) developed a prototype for a frontend interface that doctors can potentially use to monitor EHR changes/trends over time.

In the future, I aim to pursue the following: 1) Most of my work has relied on the MIMIC-IV dataset. EHRSHOT [21] (from Shah Lab at Stanford) is an extensive dataset for anonymized and structured EHR data of 6,739 patients. While my current work thus far has extensively focused on MIMIC-IV and producing time-series synthetic data, I firmly believe that using EHRSHOT (with TimeGAN, for e.g.) and adapting this

dataset to more deep learning implementations with configuration changes, data processing, etc. would allow for more reliable longitudinal trends. To this end, I am also curious about incorporating more data modalities for richer/more diverse data generation, 2) I aim to improve the frontend prototype by incorporating a data-loader where institutes/facilities can input metadata information, drag and drop their dataset, and ultimately have it seamlessly configured in the system to visualize trends through agents or other suggestions, 3) I seek to further implement the RAG and prompt engineering ideas discussed to understand how, now that we can more more feasibly generate synthetic data, we can test out a variety of LLM prompts to see which ones are optimal for extracting relevant patient information. Finally, I am also working on developing a potential approach that employs a GAN with a diffusion model (as opposed to an autoencoder with a diffusion model as I discussed for ScoEHR). The motive for this is mainly experimental as I am curious to understand how these two architectures can work together to generate high fidelity synthetic data. Ideally, in the future, I hope to implement a robust model that we can test on real patient data. The work I have done thus far for synthetic data generation enables us to move past hurdles such as extensive paperworks and training to conduct ML and scientific analyses. In our most recent meeting notes from March 13th, we also briefly discussed ideas relevant to Multimodal Neural ODEs and Document Sequence Generators (DSG), the latter of which can be used for guided text generation. I am currently reading on these topics and would be excited to implement them in the future.

7 Conclusion

Deep learning architectures such as GANs, VAEs, and diffusion models have undoubtedly paved the way in the generative data space. Through such novel architectures, we can generate synthetic electronic health records (EHRs) that protect the privacy of patients and offer high data fidelity. By generating such synthetic data, we also enable the opportunity to conduct downstream ML tasks (prediction models, classification, or in our case, prioritizing how LLMs interact with EHR data through discussed techniques such as RAG and prompt engineering). Ultimately, given that healthcare data is highly sensitive and confidential, as well as can often be unstructured from various facilities, we can leverage synthetic data to conduct meaningful analyses before we actually apply our work to real patient data.

In comparing CTGAN, SynLongEHR, a classic GAN, medGAN, and ScoEHR, it appears that CTGAN, CPAR from SynLongEHR, medGAN (with improved training tweaks), and ScoEHR are capable of producing high fidelity synthetic patient data that we can use for downstream ML tasks. CTGAN’s distribution comparison plots and PCA plots have revealed strong data fidelity for both MIMIC and EHRSHOT. For SynLongEHR, it appears that CPAR and DoppelGANger are capable of modeling similar data distributions, but could be refined for further statistical similarities to the real data. In implementing the classical GAN architecture with a comprehensive evaluation script, the synthetic output data is promising, but can be more optimal through continued training improvements and more rigorous testing with various training checkpoints. For medGAN, the model has revealed promising data generation results with an appropriate balance between the generator and discriminator in training (i.e., necessary measures such as label smoothing and adding noise were performed to enable this). Finally, for ScoEHR, although the PCA and UMAP plots reveal the need for further improvement (which could be trivial through sampling measures), the correlation matrix indicates promising synthetic patient data.

In summary, through extensive literature reviews and self-learning implementations, data processing procedures, implementations of deep learning workflows to generate synthetic data, rigorous testing to evaluate synthetic data, and a frontend user interface prototype, I gained a deep appreciation for research as a process of embracing uncertainty, experimentation, and learning. Whether it was working on a bug for hours/days or learning about an exciting deep learning implementation to generate data, I thoroughly enjoyed the research process as it has taught me to be perseverant, seek out challenges, and truly push myself to learn, experiment, and iterate.

8 Acknowledgements

I would like to sincerely take the time to thank you for reading my report. I have diligently crafted this report to showcase my results, learnings, and research journey. More importantly, thank you to [redacted] and [redacted] for their kindness and willingness to dedicate time to mentor me. I first reached out to [redacted] at the beginning of the Winter Quarter, expressing my deep interest in his multimodal research (“An Interactive Agent Foundation Model” [25]). I am privileged to have had the opportunity to pursue research with [redacted] and [redacted]. Thank you for creating a welcoming environment where I can challenge myself, learn, and grow as a researcher.

9 Literature Review and Self-Learning Continued

9.1 Towards a HIPAA Compliant Agentic AI System in Healthcare [26]

Throughout my initial discussions with [redacted] on developing an AI-powered EHR interface to visualize and understand EHR trends over time, a recurring theme that we had begun to think about was the need for privacy/security in synthetic data generation. Due to restrictions and delays in accessing large EHR datasets from sources such as UCSF, we discussed the idea of generating synthetic EHR data that could assist with our downstream ML tasks and analyses. This paper is relevant as it played an important role in motivating our concerns about generating synthetic data with high privacy and fidelity. That is, we want to ensure we can generate synthetic data that doesn't leak private information about the real dataset (i.e., we should not be able to identify the real patient from the synthetic data). Please see the methodology section where I implement privacy assessment measures.

Although this paper is not directly relevant to privacy in the synthetic data generation space, I still found it pivotal for learning about agentic frameworks for prioritizing privacy in EHR/healthcare workflows. The paper proposes a HIPAA-compliant agentic framework that consists of 3 parts: 1) Attribute-Based Access Control (ABAC) for protected health information (PHI) governance, specifically for unstructured EHR datasets, 2) a pipeline that combines regex patterns and BERT mechanisms to minimize PHI vulnerabilities/leakage, and 3) immutable audit trails for compliance verification. Such research ultimately urged greater caution in ensuring the privacy of real healthcare data, directly or through synthetic data. In my evaluation script that I discuss in the methodology section, I account for the need to inspect the privacy of the synthetic data (i.e., it should not leak real patients; this defeats the very purpose of protecting the privacy of patients). This paper is worth looking into deeper when/if we handle personally identifiable healthcare data in the future (integrating data into an interface, for instance).

9.2 ColaCare: Enhancing Electronic Health Record Modeling through Large Language Model-Driven Multi-Agent Collaboration [27]

ColaCare is a novel LLM framework that utilizes multi-agent collaboration to "bridge the gap between structured EHR data and text-based reasoning." Specifically, this paper implements a novel medical diagnoses/discussion system that leverages two main types of agents: a MetaAgent and DoctorAgents. These two types of agents cohesively collaborate with each other to foster discussions and support evidence-based clinical discussions that address "the challenge of knowledge currency." The MetaAgent manages the DoctorAgents by initiating the discussion and then facilitates several consultations/follow-ups where main concerns may be present (e.g., prediction-based tasks, evidence-dependent debates, etc.). This enables a diverse and robustly justified decision for each diagnosis/recommendation. Most importantly (and what mainly captured my motive for exploring this paper) was the incorporation of the Merck Manual of Diagnosis and Therapy (MSD) medical guideline with retrieval-augmented generation (RAG); the purpose of this was to support the medical evidence needed in relevant discussions. In my discussions with [redacted], I was introduced to the topic of RAG and had spent my time to learn about RAG's fundamentals such as the retriever, generator, dense/sparse retrieval, document chunking/preprocessing, hallucinations, and latency, to name a few (these were just a few topics I further delved into; I understand that such key terms aren't necessarily confined to the domain of RAG). In conclusion, through this paper, I gained a deeper appreciation for the use of RAG to support more accurate, context-aware responses in the medical space where evidence is above all, a very key factor in supporting medical decisions. As I describe at the end of this section, I proceeded with further self-learning about the fundamentals of RAG and the various ways they can be implemented.

9.3 From EHRs to Patient Pathways: Scalable Modeling of Longitudinal Health Trajectories with LLMs [28]

A major focus of building the EHR interface in my research experience was to ensure that we can deliver a convenient interface for enabling informed, personalized recommendations that consider a patient's trajectory over time. The motive for modeling a patient's trajectory over time was to further understand how various features would change at each visit and how such changes could potentially help with more evidence-supported diagnoses or recommendations (i.e., we can minimize the potential for uncertainty and provide more accurate recommendations when a patient comes in and we can see a continuous stream of changes over time).

In this paper, the authors highlight an important limitation on existing healthcare systems; they are very difficult to manage and interpret when it comes to "interpreting vast, heterogeneous patient data for personalized care." I found this paper interesting since it focuses on the question of how we can model patient trajectories over time (insofar we can predict future features through next time-step prediction) and how we can embody long-term temporal context into summary tokens (which could help with downstream LLM tasks).

Ultimately, the system, the longitudinal patient trajectory prediction model they build (EHR2Path), enables them to simulate patient trajectories with meaningful features such as lab results, vital signs, and LOS (length of stay). In my discussion with [redacted] and [redacted] on March 13th, we discussed the importance of making accurate patient diagnoses and recommendations by understanding each patient’s unique story. This can only be done by gaining a holistic overview of each patient’s timeline/trajectory. Thus, I found this paper to be very relevant and helpful in motivating my implementations of GANs and other deep learning architectures such as VAEs/autoencoders for synthetic data generation (as I present in the methodology section). Additionally, given that medical information can be intensely vast and diverse, the ability to model patient data over time in a seamless manner that meaningfully shows variable changes (and potential causes) can provide a tremendous benefit in healthcare diagnoses. Thus, as I show in my interface prototype, I decided to place a larger emphasis on simplistic illustrations that reveal healthcare trends over time.

9.4 Beyond Correlation: Towards Causal Large Language Model Agents in Biomedicine [29]

In this paper, the researchers from University of Missouri-Columbia reveal how large language models (LLMs) have shown great promise in the field of biomedicine broadly, but tend to lack causal skills (by defaulting to correlations). I was motivated to read this paper largely because I disagreed with their proposal that LLMs lack “true causal understanding” since, through my experience with various models from Gemini and Open AI, for instance, I was under the naive impression that causality was robustly embodied in these LLMs (perhaps through chain-of-thought reasoning, to name an example).

A pressing limitation is the fact that LLMs tend to rely on outdated and potentially unreliable data, as well as lack a true fundamental causal understanding (though, multimodal systems with LLMs could hinder the latter limitation of causality by incorporating more data sources/evidence). The paper outlines the necessary improvements we must focus on to enable greater causality in LLMs; specifically, challenges such as secure agentic frameworks, benchmarking/evals, and multimodal data integration are brought up, highlighting the possibility for multimodal data to enrich causality in LLMs, but the necessity to establish guardrails (e.g., we want to prevent agents from retrieving/processing harmful or incorrect information). The researchers also propose the idea of combining LLMs with knowledge graphs (KGs) for “grounding and formal tools” such as “MR” (model reasoning) and “DML” (declarative machine learning) for reasoning. This paper ultimately did not largely contribute to our work, but was rather explored for learning purposes/exposure to how LLMs interact with biomedical data.

9.5 Generative Large Language Models in Electronic Health Records for Patient Care Since 2023: A Systematic Review [30]

To further understand how LLMs play a role in interpreting and reasoning through EHRs, I found this paper to be particularly relevant and helpful. My main incentive for reading this paper was to focus on the key limitations present in existing literature to support further discussions on how we can use LLMs for generating and reasoning through datasets.

In this literature review, the authors discuss the various applications of leveraging generative LLMs to analyze real patient EHR data. For the topic of prompting, for example, the paper reveals that zero-shot prompting was the most common approach (used in 71/76 studies) with few-shot and chain-of-thought (CoT) prompting following, as well as other techniques such as RAG. In our meetings, [redacted] had kindly introduced me to several topics such as RAG, prompt engineering, and CoT reasoning to support my understanding of how LLMs can interact with EHR data; thus, my motivation to read this paper was to further apply this understanding. Additionally, although we did not focus on fine-tuning in this research experience, I was able to gain a more fundamental understanding of fine-tuning techniques such as LoRA and PEFT through this paper. Ultimately, the main takeaway from this paper was to survey the various methods/techniques that researchers are using for SOTA performance (e.g., prompt engineering, RAG, ChatGPT’s LLM, etc.) and apply my knowledge from our discussions by learning about the various use cases of such tools/methods (for instance, I came across several further papers that discussed ChatGPT’s aid in understanding radiology reports or even screening recommendations).

In seeking to find limitations to explore, I was fascinated by the paper’s claim on how “gender-related bias in ChatGPT’s responses” can lead to potentially unreliable use cases of AI. Furthermore, the paper reveals how GPT-J, an LLM model, has the tendency to create fictitious Human Phenotype Ontology (HPO) IDs “when addressing rare genetic diseases,” which “suggests that even advanced fine-tuning and prompting techniques may not fully eliminate the risk of hallucinations.” By exploring both the pros and cons of various ML techniques discussed (prompting, fine-tuning, etc.), I reasoned that ultimately, it is also important to generate data that is representative and diverse to ensure we don’t even provide the opportunity for biased LLM responses (although

this may be an ambitious goal); this largely influenced my work in writing comprehensive evaluation scripts for assessing the synthetic data generated (see methodology section).

9.6 Roles and Potential of Large Language Models in Healthcare: A Comprehensive Review [31]

Similar to the previous paper (3.5), the researchers in this paper highlight a recurring theme of how LLMs can provide meaningful reasoning decisions/diagnoses in medical fields. Notably, this paper highlights the limitations of LLMs in dealing with sensitive healthcare information, especially when it comes to prioritizing privacy and data security. Although LLMs have evolved from early biomedical models to now multimodal models (a successful example being Med-PaLM from Google), there are many limitations in applications such as medical examinations, clinical reasoning, patient communication/education, etc. For example, for medical examinations and clinical reasoning, the key challenges remaining are the “variable performance across different specialties” and the “need for comprehensive evaluation before clinical implementation.” Furthermore, for patient communication/education, the key challenges are “readability levels often [exceeding] patient comprehension” and the need to ensure “consistency with medical guidelines” (the latter of which can frankly be improved with RAG).

I found Table 2 (how to mitigate hallucinations) of this paper to be most relevant and helpful to our interests on understanding how we can leverage LLMs with EHR data. This table clearly contrasts the benefits and limitations of techniques such as prompt engineering, RAG, and fine-tuning (all of which are known to hinder the possibility of hallucinations).

In developing an EHR interface, I found this paper to be helpful as it supported my decision to initially experiment with off-the-shelf models such as OpenAI (ChatGPT) and Anthropic (Sonnet 3.7) to reason through the EHR data. Although I did not rigorously explore fine-tuning multimodal models or LLMs, I enjoyed learning about how we can employ techniques such as prompting and fine-tuning to further improve the performance of LLMs to pivot their general-purpose capabilities to a highly specific medical domain/context.

9.7 Enhancing medical AI with retrieval-augmented generation: A mini narrative review [32]

As discussed in the previous surveys of papers, a primary topic that had come up in our discussions was retrieval-augmented generation (RAG) and the potential for it to embody LLMs with more context-aware and potentially accurate outputs. Alongside this paper, I self-studied the RAG fundamentals through YouTube and online tutorials (Hugging Face, LangChain). In particular, I found this tutorial on building your own RAG from scratch to be very helpful in gaining intuition on the retrieval function.

In supporting my further learning and reading on RAGs applications in medicine, I came across this paper. The paper highlights various use cases of RAG in medicine, ranging from “diagnostic assistance” to “clinical information retrieval.” Moreover, as I had come across papers that largely focused on the limitations/challenges RAG when it came to mitigating hallucinations (take, for instance, paper 3.6 above), I found this report to be quite helpful in providing solutions to the previously mentioned challenges (e.g., complex integration, outdated/irrelevant information). Particularly, I learned about recommendations on optimizing the retrieval mechanisms and embedding models as potential next steps.

For initial prototypes with the EHR dashboard interface, I had chosen to proceed with using OpenAI’s GPT-4 model as a first step. This decision was motivated by the study’s support for GPT-4 since “RAG alone notably improved GPT-4’s performance (AUC 0.82, AUPRC 0.56, accuracy 81.3%),” which highlighted the LLM’s potential to be significantly contextualized for medical-specific datasets (and perhaps offer more comprehensive, accurate, and tailored responses to EHR-specific queries).

9.8 A Scalable Method for Validated Data Extraction from Electronic Health Records with Large Language Models [33]

In anticipating the potential challenge of learning how to most effectively allow LLM models to extract relevant contextual information from EHR data, I had chosen to read this paper. As I learned through the previous papers that LLMs can interpret and reason rich information through several modalities (which can support greater causality), the question of how to extract, interpret, and reason through such data became evidently important.

In this paper, the researchers outline a clear, step-by-step approach on extracting and processing EHR data through LLMs for various downstream ML tasks such as classification, prediction, etc. Firstly, the step of data collection and preprocessing is employed; patient documents in various formats (semistructured, different modalities, etc.) are received. I found this first point, although not very unintuitive, to be a good reminder of the need to build a scalable, generalizable pipeline in the future. That is, most of the synthetic data I have generated is well structured, whereas in the real world, a lot of institutions/hospitals often have large,

vast, and unstructured databases. For preprocessing, document parsing, optical character recognition, removing outliers/missing values, etc. are key steps. After this, appropriate formatting decisions are made depending on the dataset (e.g., could be dividing the document into specific chunks with respect to the C-CDA format), a text embedding model is used to embed the information into a vector representation (we want to capture the semantic content in a rich latent space). Finally, we can proceed with our ML tasks such as classification and labeling. Although this is a high-level overview of how to approach the use of LLMs with EHR datasets, I nonetheless found this paper to be very helpful in guiding various strategies for data preprocessing/preparation.

9.9 Using large language models to extract PD-L1 biomarker data from electronic health records [34]

In this paper, published by researchers at Flatiron Health, a biotechnology research company, I learned about the specific use case of pre-trained LLMs in extracting PD-L1 biomarker data from EHRs and fine-tuning to support domain-specific knowledge. By fine-tuning the LLM with clearly labeled data, they had concluded that the LLMs could outperform the general zero-shot approach. Although this wasn't very surprising, I was excited to learn that the fine-tuned LLMs could outperform a deep-learning architecture trained on 10,000 examples (as opposed to the minimal examples in the range of 500-1500 that were used for fine-tuning). In my research, although I didn't experiment with fine-tuning, this prompted further thinking on how we must be careful to prevent overfitting and whether or not there is a "sweet spot" per se of fine-tuning models, or training broadly, with labeled data to allow for optimal accuracy.

While this paper focused specifically on biomarker results, it was still important and relevant to my learning on fine-tuning LLMs with multiple data modalities (e.g., in future work, we could focus on extensive modalities as opposed to the specific use case of biomarkers).

9.10 Self-learning Summarized

Now that I have discussed a few papers that I came across in my initial weeks of research, I will be summarizing a few brief learnings (i.e., with respect to various topics such as RAG, prompt-engineering, etc.) below. In self-learning about these topics, I leveraged YouTube tutorial videos (Andrej Karpathy's *Zero to Hero Playlist*, *3Blue1Brown's Neural Network Playlist*), online blogs (*Medium*, *Hugging Face*, *Towards Data Science*), and publications to gain more exposure (as I have discussed above). Note that I do not exclusively put all of my learnings here to avoid redundancy as they are also present in the methodology section (where I discuss my learnings about architectures such as GANs and autoencoders as I implemented them). Brief learning notes from initial videos watched linked here.

In learning about RAG, I found Mistral AI's tutorial on building a RAG from scratch to be helpful for gaining intuition, processing document chunks, creating embeddings, enabling queries to generate context-aware responses, and the various ways RAG can be implemented (e.g., through Mistral, LangChain, LlamaIndex, etc.). In an effort to provide a more measurable outcome, I have revisited this tutorial and applied the RAG tutorial to leverage information from PubMed abstracts (specifically, through Entrez Programming). By identifying relevant abstracts based on a given context (in this case, 20 abstracts with the hint "diabetes treatment methods"), I was able to create a simple RAG system that could provide insights to the query, "what are common symptoms of diabetes?" Code linked here.

For prompt engineering, besides experimenting with how different prompts/context-specific hints would enable more descriptive responses through the RAG tutorial mentioned, I found the following tutorial from Anthropic to be exciting for learning about system prompts, which are effectively guided hints or instructions on how the system must reply to the user. Take, for instance, the following example from the notebook: `SYSTEM_PROMPT = "Your answer should always be a series of critical thinking questions that further the conversation (do not provide answers to your questions). Do not actually answer the user's question."` While I did not directly apply these towards a medical application, I had learned about the basics of prompt engineering through this notebook and had brainstormed how I could embody specific queries with strict instructions (i.e., looking at a given patient's vital signs over time, what can you infer, etc. to focus the attention of the LLM when experimenting with the user interface prototype).

Lastly, for fine-tuning, I had particularly explored short tutorials such as "Fine-Tuning Your First Large Language Model (LLM) with PyTorch and Hugging Face" from HuggingFace and starter-videos from Google Cloud Tech's YouTube channel (e.g., this video about fine-tuning Gemini). Although I did not fine-tune any models, I found these resources important in gaining an intuition for techniques such as LoRA and how they truly work by freezing certain parameters, formatting datasets in various structures (conversational, instruction-based, etc.), and how to deal with encoding/decoding multiple data modalities (e.g., VLMs such as PaliGemma [35] use SigLIP for encoding and Gemma for decoding).

References

- [1] Google DeepMind. *AlphaFold*. Mar. 12, 2025. URL: deepmind.google/technologies/alphafold (visited on 03/12/2025).
- [2] John Jumper et al. “Highly Accurate Protein Structure Prediction With AlphaFold”. In: *Nature* 596.7873 (July 2021), pp. 583–589. DOI: 10.1038/s41586-021-03819-2. URL: <https://www.nature.com/articles/s41586-021-03819-2#citeas>.
- [3] Kevin Black et al. π_0 : A Vision-Language-Action Flow Model for General Robot Control. 2024. arXiv: 2410.24164 [cs.LG]. URL: <https://arxiv.org/abs/2410.24164>.
- [4] Michelle Kang Kim et al. “Challenges in and Opportunities for Electronic Health Record-Based Data Analysis and Interpretation”. In: *Gut and Liver* 18.2 (Oct. 2023), pp. 201–208. DOI: 10.5009/gnl230272. URL: <https://doi.org/10.5009/gnl230272>.
- [5] Jiancheng Ye et al. *The Role of Artificial Intelligence for the Application of Integrating Electronic Health Records and Patient-generated Data in Clinical Decision Support*. May 2024. URL: <https://pmc.ncbi.nlm.nih.gov/articles/PMC11141850/>.
- [6] Vasileios C. Pezoulas et al. “Synthetic data generation methods in healthcare: A review on open-source tools and methods”. In: *Computational and Structural Biotechnology Journal* 23 (2024), pp. 2892–2910. ISSN: 2001-0370. DOI: 10.1016/j.csbj.2024.07.005. URL: <https://www.sciencedirect.com/science/article/pii/S2001037024002393>.
- [7] Chao Yan et al. “Generating Synthetic Electronic Health Record Data Using Generative Adversarial Networks: A Tutorial (Preprint)”. In: *JMIR AI* 3 (Mar. 2024), e52615. DOI: 10.2196/52615. URL: <https://pubmed.ncbi.nlm.nih.gov/38875595/>.
- [8] Lei Xu et al. “Modeling tabular data using conditional GAN”. In: *Proceedings of the 33rd International Conference on Neural Information Processing Systems*. Red Hook, NY, USA: Curran Associates Inc., 2019.
- [9] Jim L. Achterberg, Marcel R. Haas, and Marco R. Spruit. “On The Evaluation of Synthetic Longitudinal Electronic Health Records”. In: *BMC Medical Research Methodology* 24.1 (Aug. 2024). DOI: 10.1186/s12874-024-02304-4. URL: <https://doi.org/10.1186/s12874-024-02304-4>.
- [10] Zinan Lin et al. “Using GANs for Sharing Networked Time Series Data: Challenges, Initial Promise, and Open Questions”. In: *Proceedings of the ACM Internet Measurement Conference*. IMC ’20. ACM, Oct. 2020, 464–483. DOI: 10.1145/3419394.3423643. URL: <http://dx.doi.org/10.1145/3419394.3423643>.
- [11] Kevin Zhang, Neha Patki, and Kalyan Veeramachaneni. *Sequential Models in the Synthetic Data Vault*. 2022. arXiv: 2207.14406 [cs.LG]. URL: <https://arxiv.org/abs/2207.14406>.
- [12] Edward Choi et al. *Generating Multi-label Discrete Patient Records using Generative Adversarial Networks*. 2018. arXiv: 1703.06490 [cs.LG]. URL: <https://arxiv.org/abs/1703.06490>.
- [13] Aa Naseer et al. *ScoEHR: Generating Synthetic Electronic Health Records Using Continuous-time Diffusion Models*. 2023. URL: <https://ora.ox.ac.uk/objects/uuid:0f3aef8d-efd3-4564-8d4a-51974970a083>.
- [14] Alistair E. W. Johnson et al. “MIMIC-IV, a Freely Accessible Electronic Health Record Dataset”. In: *Scientific Data* 10.1 (Jan. 2023). DOI: 10.1038/s41597-022-01899-x. URL: <https://www.nature.com/articles/s41597-022-01899-x#citeas>.
- [15] Alistair Johnson et al. *MIMIC-IV (version 3.1)*. RRID:SCR_007345. 2024. DOI: 10.13026/kpb9-mt58. URL: <https://doi.org/10.13026/kpb9-mt58>.
- [16] Alistair E.W. Johnson et al. “MIMIC-III, a Freely Accessible Critical Care Database”. In: *Scientific Data* 3.1 (May 2016). DOI: 10.1038/sdata.2016.35. URL: <https://pmc.ncbi.nlm.nih.gov/articles/PMC4878278/>.
- [17] Stephanie L. Hyland et al. “Early Prediction of Circulatory Failure in the Intensive Care Unit Using Machine Learning”. In: *Nature Medicine* 26.3 (Mar. 2020), pp. 364–373. DOI: 10.1038/s41591-020-0789-4. URL: <https://doi.org/10.1038/s41591-020-0789-4>.
- [18] Tom J. Pollard et al. “The eICU Collaborative Research Database, a Freely Available Multi-center Database for Critical Care Research”. In: *Scientific Data* 5.1 (Sept. 2018). DOI: 10.1038/sdata.2018.178. URL: <https://doi.org/10.1038/sdata.2018.178>.
- [19] Srija Chaturvedula, Yaswanth Battineedi, and IRJET. *Comparative Analysis of GANs and VAEs in Generating High-Quality Images: A Case Study on the MNIST Dataset*. Tech. rep. 01. 2024, pp. 485–. URL: <https://www.irjet.net/archives/V11/i1/IRJET-V11I178.pdf>.

- [20] Rohit Sahoo et al. *GANs and VAEs as Methods of Synthetic Data Generation and Augmentation to Enhance Heart Disease Prediction*. Tech. rep. 2. Dec. 2021, pp. 17–18. DOI: 10.35940/ijeat.B3263.1211221. URL: <https://www.ijeat.org/wp-content/uploads/papers/v11i2/B32631211221.pdf>.
- [21] Michael Wornow et al. *EHRSHOT: An EHR Benchmark for Few-Shot Evaluation of Foundation Models*. 2023. arXiv: 2307.02028 [cs.LG]. URL: <https://arxiv.org/abs/2307.02028>.
- [22] Ian J. Goodfellow et al. *Generative Adversarial Networks*. 2014. arXiv: 1406.2661 [stat.ML]. URL: <https://arxiv.org/abs/1406.2661>.
- [23] et al. “Precision population analytics: population management at the point-of-care”. In: *Journal of the American Medical Informatics Association : JAMIA* 28.3 (2021), pp. 588–595. DOI: 10.1093/jamia/ocaa247.
- [24] Adam Tauman Kalai et al. *Why Language Models Hallucinate*. 2025. arXiv: 2509.04664 [cs.CL]. URL: <https://arxiv.org/abs/2509.04664>.
- [25] et al. *An Interactive Agent Foundation Model*. 2024. arXiv: 2402.05929 [cs.AI]. URL: <https://arxiv.org/abs/2402.05929>.
- [26] Subash Neupane, Sudip Mittal, and Shahram Rahimi. *Towards a HIPAA Compliant Agentic AI System in Healthcare*. 2025. arXiv: 2504.17669 [cs.MA]. URL: <https://arxiv.org/abs/2504.17669>.
- [27] Zixiang Wang et al. “ColaCare: Enhancing Electronic Health Record Modeling through Large Language Model-Driven Multi-Agent Collaboration”. In: *Proceedings of the ACM on Web Conference 2025*. WWW ’25. ACM, Apr. 2025, 2250–2261. DOI: 10.1145/3696410.3714877. URL: <http://dx.doi.org/10.1145/3696410.3714877>.
- [28] Chantal Pellegrini et al. *From EHRs to Patient Pathways: Scalable Modeling of Longitudinal Health Trajectories with LLMs*. 2025. arXiv: 2506.04831 [cs.LG]. URL: <https://arxiv.org/abs/2506.04831>.
- [29] Adib Bazgir, Amir Habibdoust Lafmajani, and Yuwen Zhang. *Beyond Correlation: Towards Causal Large Language Model Agents in Biomedicine*. 2025. arXiv: 2505.16982 [cs.AI]. URL: <https://arxiv.org/abs/2505.16982>.
- [30] Xinsong Du et al. “Generative Large Language Models in Electronic Health Records for Patient Care Since 2023: A Systematic Review”. In: *medRxiv* (2024). DOI: 10.1101/2024.08.11.24311828. eprint: <https://www.medrxiv.org/content/early/2024/08/19/2024.08.11.24311828.full.pdf>. URL: <https://www.medrxiv.org/content/early/2024/08/19/2024.08.11.24311828>.
- [31] Chihung Lin and Chang-Fu Kuo. “Roles and Potential of Large Language Models in Healthcare: A Comprehensive Review”. In: *Biomedical Journal* (2025), p. 100868. ISSN: 2319-4170. DOI: <https://doi.org/10.1016/j.bj.2025.100868>. URL: <https://www.sciencedirect.com/science/article/pii/S2319417025000423>.
- [32] Omid Kohandel Gargari and Gholamreza Habibi. “Enhancing Medical AI With Retrieval-augmented Generation: A Mini Narrative Review”. In: *Digital Health* 11 (Apr. 2025). DOI: 10.1177/20552076251337177. URL: <https://pmc.ncbi.nlm.nih.gov/articles/PMC12059965/>.
- [33] Timothy J. Stuhlmiller et al. “A Scalable Method for Validated Data Extraction from Electronic Health Records with Large Language Models”. In: *medRxiv* (2025). DOI: 10.1101/2025.02.25.25322898. eprint: <https://www.medrxiv.org/content/early/2025/02/26/2025.02.25.25322898.full.pdf>. URL: <https://www.medrxiv.org/content/early/2025/02/26/2025.02.25.25322898>.
- [34] Flatiron Health. *Using large language models to extract PD-L1 biomarker data from electronic health records*. Feb. 2025. URL: <https://resources.flatiron.com/publications/using-large-language-models-to-extract-pd-l1-biomarker-data-from-electronic-health-records>.
- [35] Lucas Beyer et al. *PaliGemma: A versatile 3B VLM for transfer*. 2024. arXiv: 2407.07726 [cs.CV]. URL: <https://arxiv.org/abs/2407.07726>.